

MethylPipeline Theory

Mathematical foundations, assumptions, and limitations

MethylPipeline Team

2026-07-07

Table of contents

1	Scope, Traceability, and Notation	7
1.1	Purpose	7
1.2	What Counts As “Theory” In This Repository	7
1.3	Package Map	8
1.4	Shared Data Model	9
1.5	Shared Notation	9
1.6	Cross-Cutting Assumptions	10
1.7	Reading This Book	10
1.7.1	Package Map	10
I	Part I: Mathematical and Statistical Foundations	12
2	MethylUtils	13
2.1	Scope	13
2.2	Implemented Method Inventory	13
2.3	Centroid Sufficient Statistics	14
2.4	ECDF Reconstruction	14
2.5	Two-Sample Comparison	15
2.5.1	KS Statistic on Reconstructed ECDFs	15
2.5.2	Mann-Whitney from Histogram Counts	15
2.5.3	Heterogeneity	16
2.6	Continuous Overlap	16
2.7	Multiple Testing and p-Value Aggregation	16
2.8	Canonical Biological Effect Size	17
2.9	ECDFClassifier	17
2.10	Auxiliary Beta-Based Metrics	18
3	MethylCentroid	19
3.1	Role	19
3.2	What A Centroid Stores	19
3.3	Sample-To-Centroid Update Rule	19
3.4	Coverage Capping By Binomial Thinning	20
3.5	Auto-Cap Heuristic	20
3.6	Why Histograms Matter	21
3.7	Publication Guidance	21
4	MethylDetector	22
4.1	Role	22
4.2	Alignment And Candidate Set	22

4.3	Statistical Screening	22
4.4	Biological Ranking	23
4.5	Cumulative Effect Coverage	23
4.6	Context And Chromosome Granularity	23
4.7	Exported Classifier Artifacts	24
4.8	Discovery versus prediction exports	24
4.9	Production Freeze Panel	24
4.10	DMP Coverage in New Samples	25
4.11	Publication Guidance	26
5	MethylClassifier	27
5.1	Role	27
5.2	Binary Classification	27
5.3	Multi-Chromosome Binary Classification	27
5.4	One-vs-Rest (OvR) Multiclass Fusion	28
5.5	Pairwise Control Aggregation	28
5.6	Bipartite Multi-Control $\times$ Multi-Disease Aggregation	29
5.7	Calibration	29
5.8	New Modeling Features in Production	29
5.8.1	Learned Multiclass Head	29
5.8.2	Isotonic and Stacking Layers	30
5.8.3	Practical Reproducibility	30
5.9	Multiclass Learned Head: Class Weighting	30
5.10	Missing Data Semantics	31
5.11	Publication Guidance	31
6	MethylPredictor, MethylValidation, and MethylDiseaseProgression	32
6.1	Role	32
6.2	MethylPredictor: Labeled Evaluation	32
6.3	Blind Prediction Summaries	33
6.4	MethylValidation: Split Strategy	33
6.5	Monte Carlo Evaluation	33
6.6	What The Validation Layer Does Not Do	34
6.7	Stability Analysis	34
6.8	Production Freeze	35
6.9	Hybrid Generative Backend (<code>model_backend=generative_hybrid</code>)	35
6.9.1	Data Contract and Bundle Reuse	35
6.9.2	Modeling Structure	36
6.9.3	Orchestration in <code>--model</code>	36
6.9.4	Configuration Surface	36
6.10	MethylDiseaseProgression Synthesis	37
6.10.1	Inputs and Stage Resolution	37
6.10.2	Entity Scores	37
6.10.3	Progression Labels	37
6.10.4	Outputs	38
6.10.5	Optional gene set overlap metrics	38
6.10.6	Stage-wise enrichment interpretation (prostate cancer template)	38
6.10.7	Key axes of progression (what is changing)	39

6.11	Two Workflows Summary	40
6.12	Publication Guidance	40
7	MethylMapper	41
7.1	Role	41
7.2	Interval Mapping	41
7.3	Gene-Level Combined p-Values	41
7.4	Biological Importance Score	42
7.5	Stable Gene Set Optimization	42
7.6	External Evidence Enrichment	42
7.7	Publication Guidance	43
7.8	Mapper As Biological Validation Layer	43
8	MethylEnricher	44
8.1	Role	44
8.2	Over-Representation Analysis	44
8.2.1	Library sets used by MethylEnricher	44
8.3	Pathway Graph Construction	45
8.4	Why This Layer Matters For DMP Validation	46
8.5	Module Enrichment Score	46
8.6	Disease Relevance Prior	47
8.7	Optional PPI Network Refinement	47
8.8	Visualization Modes	48
8.9	Project-Orchestrated Execution	48
8.10	Theme Normalization And Labels	48
8.11	Publication Guidance	48
9	MethylAlignmentQC	49
9.1	Role	49
9.2	Input metrics	49
9.2.1	Picard-style deduplication metrics	49
9.2.2	Parabricks WGBS metrics JSON	49
9.2.3	Alignment-layer guardrails	50
9.3	Export JSON (V2)	50
9.4	Cycle screening and remediation	50
9.5	cfDNA fragmentomics (optional)	51
9.6	Summary statistics	51
9.7	Publication guidance	51
9.8	Related	51
10	MethylExtractionQC	52
10.1	Role	52
10.2	Input: extraction manifest	52
10.3	Guardrails	52
10.4	Configuration	53
10.5	Why both alignment and extraction QC	53
10.6	Publication guidance	53
10.7	Related	53

11	Limitations and Open Questions	54
11.1	Why This Chapter Exists	54
11.2	The “ECDF-Only Pipeline” Claim Is Too Broad	54
11.3	Dependence Is Real	54
11.4	Approximation Layers	54
11.5	Heuristic Layers	55
11.6	External Dependencies	55
11.7	Legacy Or Experimental Remnants	55
11.7.1	Current safeguards	56
11.7.2	Deprecation Status Snapshot	56
11.8	Publication-Ready Positioning	56
11.9	What Would Strengthen The Theory Further	57
II	Part II: Configuration Semantics and Workflow Theory	58
12	The Project Configuration File	59
12.1	Role and design principle	59
12.2	Four-layer configuration	59
12.3	Top-level study manifest fields	59
12.4	The cohort hierarchy	60
12.5	Where tool parameters live: <code>actionConfig</code>	61
12.6	Monte Carlo configuration	61
12.7	Annotated study manifest example	62
12.8	Output directory layout	63
12.9	Related	63
13	The Two Workflows	64
13.1	Overview	64
13.1.1	DMP and gene modeling modes	64
13.2	Workflow 1: Model Creation	65
13.2.1	Theoretical Motivation	65
13.2.2	The Three Stages of Workflow 1	65
13.2.3	Steps	67
13.2.4	Workflow 1 Diagram	69
13.2.5	Expected Outputs	69
13.3	Workflow 2: Model Use for Prediction	69
13.3.1	Theoretical Motivation	69
13.3.2	Steps	71
13.3.3	Workflow 2 Diagram	71
13.4	Workflow 3: True Held-Out Batch Evaluation	71
13.4.1	Theoretical Motivation	71
13.4.2	Designating the Held-Out Batch	72
13.4.3	Disjointness Guarantee (Exclusion at Freeze)	73
13.4.4	Bootstrap Distribution of QC Metrics	73
13.4.5	Steps	73
13.4.6	Caveat	74
13.5	Diagnostic: DMP Coverage	74

13.6	Diagnostic: Class Imbalance	75
13.7	Diagnostic: Sklearn Version Mismatch	75
13.8	Choosing Workflow Parameters	76
13.8.1	<code>train_fraction</code>	76
13.8.2	<code>n_iterations</code>	76
13.8.3	<code>stability_dmp_freq</code>	76
13.8.4	<code>stability_min_balanced_accuracy</code>	76
14	Model Creation and Validation	77
14.1	Scope	77
14.2	Training and Validation Lifecycle	77
14.3	Backend Roles	78
14.4	Covariate-Aware Validation Contract	78
14.5	Outputs to Review	79
14.6	Configuration Audit for Accuracy-First Modeling	79
14.6.1	Deprecation candidates (proposed)	79
14.6.2	Accuracy-first project profile	80
14.6.3	Reference tuning template (binary ECDF-first)	80
14.7	Distributed Monte Carlo on shared storage	81
14.8	Optional pipeline hyperparameter search	81

1 Scope, Traceability, and Notation

1.1 Purpose

This book documents the mathematical and statistical behavior that MethylPipeline actually implements. It is not a historical reconstruction of the first design notes, and it is not a promise about future refactors. The source of truth is the current code under `packages/`.

That rule has two consequences:

1. If the code and an older markdown document disagree, the code is canonical.
2. If the code uses an approximation or heuristic, the book says so explicitly instead of rewriting it into a cleaner but false theory story.

1.2 What Counts As “Theory” In This Repository

The repository mixes several kinds of methodology. For publication-quality documentation it is important to separate them.

Category	Meaning in this book	Typical examples in MethylPipeline
Principled	Standard statistical or numerical methods used in a recognizable form	asymptotic Kolmogorov-Smirnov testing, Storey q-values, weighted log-likelihood classification, classical clustering algorithms
Approximate	A standard method implemented through discretization, asymptotics, or reconstruction from summaries rather than raw observations	histogram-based Mann-Whitney, grid-based ECDF overlap, effective-sample-size KS p-values
Heuristic	Engineered ranking, thresholding, or aggregation rules chosen for workflow utility rather than statistical optimality	effect-size ranking, module scoring constants, pairwise control aggregation

Category	Meaning in this book	Typical examples in MethylPipeline
External-service-backed	Results depend on external databases, APIs, or stored procedures whose internal logic is not fully versioned inside this repository	Enrichr, DisGeNET, Open Targets, Azure SQL stored procedures, LLM-assisted evidence synthesis

That taxonomy is used throughout the book so the reader can tell where the pipeline is closest to a classical statistical method and where it is best interpreted as a reproducible engineering decision.

1.3 Package Map

The complete package map with all four parts is in the “Reading This Book” section below. For quick orientation, Part I covers the mathematical and statistical algorithms in the following packages:

Chapter	Primary packages	Role
Chapter 2	<code>methyutils</code>	shared mathematical layer: centroids, ECDF views, testing, effect sizes, classifier densities
Chapter 3	<code>methylcentroid</code>	centroid construction and coverage preprocessing
Chapter 4	<code>methyldetector</code>	DMP testing, filtering, classifier export, and production panel bypass
Chapter 5	<code>methylclassifier</code>	binary, multi-chromosome, OvR, and multiclass learned-head classification
Chapter 6	<code>methylpredictor</code> , <code>methylvalidation</code> , <code>methyliseaseprogression</code>	prediction metrics, Monte Carlo validation, stability analysis, production freeze, progression synthesis
Chapter 7	<code>methylmapper</code>	DMP-to-gene mapping and gene-level aggregation
Chapter 8	<code>methylenricher</code>	over-representation analysis, pathway graph clustering, and module scoring
Chapter 9	<code>methylalignmentqc</code>	deterministic QC parsing and summarization
Chapter 11	<code>all</code>	caveats, legacy remnants, and claims that should be softened

1.4 Shared Data Model

At the lowest level the code works with per-position methylated and unmethylated counts. For a sample s at genomic position i ,

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m_{si} is the methylated count and u_{si} is the unmethylated count. The code clips or guards zero-coverage positions instead of leaving them undefined.

For a centroid with N_i contributing samples at locus i , the shared sufficient statistics are

$$N_i = \sum_s 1, \quad S_{x,i} = \sum_s x_{si}, \quad S_{x^2,i} = \sum_s x_{si}^2.$$

These yield the implemented centroid mean and unbiased sample variance:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right),$$

with a small numerical floor ε used throughout the code to avoid degenerate densities or divisions by zero.

In the ECDF-centered path, each locus also stores a binned histogram

$$\{c_{ib}\}_{b=1}^B$$

over shared methylation bin edges $0 = e_0 < e_1 < \dots < e_B = 1$. Those histograms drive both empirical distribution reconstruction and approximate rank-based testing.

1.5 Shared Notation

Symbol	Meaning
x_{si}	methylation fraction for sample s at locus i
m_{si}, u_{si}	methylated and unmethylated counts
$N_i, S_{x,i}, S_{x^2,i}$	centroid sufficient statistics
c_{ib}	count in histogram bin b for locus i
$F_i(x)$	reconstructed cumulative distribution function at locus i
$f_i(x)$	reconstructed density, usually the derivative of a monotone spline
D_i	two-sample KS statistic at locus i
U_i	Mann-Whitney U statistic at locus i
q_i	Storey-adjusted q-value

Symbol	Meaning
O_i	continuous overlap $\int_0^1 \min(f_{1i}, f_{2i}) dx$
w_i	feature weight used by the classifier, usually derived from detector output

1.6 Cross-Cutting Assumptions

Several assumptions recur across the repository:

- Loci are treated as conditionally independent inside the classifier score, even though methylation along the genome is correlated.
- Multiple-testing corrections are applied across many dependent loci and, downstream, across aggregated genes. This is standard practice but should not be oversold as exact finite-sample control.
- Some packages operate on summaries rather than raw per-read data. That preserves scalability, but it changes the interpretation of rank tests and distribution overlap.
- Downstream packages use external services and curated priors. Those are part of the workflow, but they are not internal statistical estimators.

1.7 Reading This Book

This book is organized into two parts (mathematical foundations and workflow/configuration theory). CLI runbooks, deployment, and exhaustive parameter tables were moved to the **Usage** and **Reference** pillars during the 2026-06 documentation IA revision.

Part I: Mathematical and Statistical Foundations covers the algorithms and estimators in order from the lowest shared layer to the downstream applications. Readers interested in the production classifier should start with Chapter 2, then continue through Chapter 3, Chapter 4, and Chapter 5. Readers interested in biological interpretation should continue to Chapter 7 and Chapter 8. Chapter 11 is intentionally blunt and should be read before any publication claim is finalized.

Part II: Configuration Semantics and Workflow Theory documents study manifest anatomy (Chapter 12), the two operational workflow models (Chapter 13), and model creation/validation theory (Chapter 14).

Operator commands and deployment live in the [Usage manual](#). **Parameter lookup** lives in [Reference: configuration-reference](#). **Implementation internals** live in [Implementation guide](#).

1.7.1 Package Map

Part	Chapter	Primary packages	Role
I	Chapter 2	<code>methylutils</code>	shared mathematical layer
I	Chapter 3	<code>methylcentroid</code>	centroid construction
I	Chapter 4	<code>methyldetector</code>	DMP testing and panel export
I	Chapter 5	<code>methylclassifier</code>	classification heads
I	Chapter 6	<code>methylpredictor</code> , <code>methylvalidation</code> , <code>methyliseaseprogression</code>	validation and freeze theory
I	Chapter 7	<code>methylmapper</code>	DMP-to-gene mapping
I	Chapter 8	<code>methylenricher</code>	enrichment and modules
I	Chapter 9	<code>methylalignmentqc</code>	QC parsing
I	Chapter 11	<code>all</code>	caveats and legacy
II	Chapter 12	<code>methylutils</code> (ProjectConfig)	study manifest semantics
II	Chapter 13	<code>all</code>	workflow theory
II	Chapter 14	<code>methylvalidation</code>	model lifecycle theory

Part I

Part I: Mathematical and Statistical Foundations

2 MethylUtils

2.1 Scope

`methylutils` is the mathematical spine of the repository. The core implementations live in:

- `methyl_utils/core/centroid_builder.py`
- `methyl_utils/core/distribution_views.py`
- `methyl_utils/methyl_centroid_pair.py`
- `methyl_utils/statistical_tests.py`
- `methyl_utils/ecdf_classifier.py`

Those modules define the empirical-distribution view used by centroid comparison, DMP detection, and ECDF classification. They also expose several auxiliary or legacy methods, especially beta-based distances, that remain available as optional utilities.

2.2 Implemented Method Inventory

Area	Main code path	Status in this book
centroid sufficient statistics	<code>core/centroid_builder.py</code>	principled
ECDF reconstruction from histograms	<code>core/distribution_views.py</code>	principled numerical representation
continuous overlap integral	<code>statistical_tests.py</code>	approximate numerical integration
KS testing on reconstructed ECDFs	<code>statistical_tests.py</code>	asymptotic and grid-based
Mann-Whitney from binned counts	<code>statistical_tests.py</code>	approximate because it reconstructs rank information from histograms
Storey q-values	<code>statistical_tests.py</code>	principled, with standard tuning caveats
DerSimonian-Laird style heterogeneity	<code>statistical_tests.py</code>	principled method-of-moments adaptation
canonical biological effect size	<code>statistical_tests.py</code>	heuristic ranking score
ECDFClassifier	<code>ecdf_classifier.py</code>	principled scoring rule plus heuristic stabilizers
beta distances and overlaps	<code>metrics_core.py</code>	auxiliary; not the main detector-classifier path

2.3 Centroid Sufficient Statistics

For each locus i , the builder stores per-sample methylation fractions x_{si} together with aggregate statistics:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right), \quad (2.1)$$

where N_i is the number of samples contributing to that locus and ε is a small numerical floor.

The centroid builder also stores count-level summaries:

$$S_{m,i} = \sum_s m_{si}, \quad S_{u,i} = \sum_s u_{si}, \quad S_{c^2,i} = \sum_s (m_{si} + u_{si})^2, \quad S_{wx^2,i} = \sum_s \frac{m_{si}^2}{m_{si} + u_{si}}, \quad (2.2)$$

which are later reused by the heterogeneity calculation. Finally, each locus accumulates histogram counts c_{ib} over shared bin edges $\{e_b\}$, producing the `binned_stats` object that drives both ECDF reconstruction and histogram-based testing.

2.4 ECDF Reconstruction

The code constructs a cumulative distribution at bin edges by normalizing cumulative bin counts:

$$\tilde{F}_i(e_b) = \frac{\sum_{r \leq b} c_{ir}}{\sum_{r=1}^B c_{ir}}, \quad b = 0, \dots, B. \quad (2.3)$$

`ECDFView` then interpolates these edge values with a monotone PCHIP spline ([Fritsch and Carlson 1980](#)). The resulting representation supplies:

- a continuous cumulative distribution $\tilde{F}_i(x)$,
- a continuous density proxy $\tilde{f}_i(x) = \max(\partial_x \tilde{F}_i(x), \varepsilon)$, and
- numerically stable means and variances via the stored sufficient statistics.

This is an empirical-distribution model, not a beta or Gaussian fit. That distinction is central to the detector-classifier path.

2.5 Two-Sample Comparison

2.5.1 KS Statistic on Reconstructed ECDFs

For aligned centroid pairs, the code evaluates the two-sample Kolmogorov-Smirnov statistic

$$D_i = \sup_{x \in [0,1]} \left| \tilde{F}_{1i}(x) - \tilde{F}_{2i}(x) \right|. \quad (2.4)$$

Implementation detail matters here: the supremum is approximated on a uniform grid rather than computed analytically, and the p -value is obtained from the large-sample Smirnov distribution ([Kolmogorov 1933](#); [Feller 1948](#)) using the harmonic mean effective sample size

$$n_{\text{eff},i} = \frac{2}{1/\max(n_{1i}, 1) + 1/\max(n_{2i}, 1)}, \quad p_i = \Pr(K \geq \sqrt{n_{\text{eff},i}} D_i). \quad (2.5)$$

This is a reasonable screening approximation, but it is not an exact finite-sample KS test on raw observations.

2.5.2 Mann-Whitney from Histogram Counts

The repository also implements a vectorized Mann-Whitney calculation directly from centroid histograms. Let $c_{ib}^{(1)}$ and $c_{ib}^{(2)}$ be the two groups' bin counts at locus i . The code reconstructs the U statistic as

$$U_i = \sum_{b=1}^B c_{ib}^{(1)} \left(\sum_{r < b} c_{ir}^{(2)} + \frac{1}{2} c_{ib}^{(2)} \right). \quad (2.6)$$

This is followed by the usual normal approximation,

$$\mu_{U,i} = \frac{n_{1i}n_{2i}}{2}, \quad Z_i = \frac{U_i - \mu_{U,i}}{\sqrt{\text{Var}(U_i)}},$$

with a tie correction derived from the combined histogram counts. The important caveat is that the code does **not** reconstruct the full rank ordering of raw methylation values; it reconstructs a rank-like summary from histogram bins. In this book that is treated as an approximation to the classical Mann-Whitney test ([Mann and Whitney 1947](#)), not as an identical procedure.

2.5.3 Heterogeneity

`dl_heterogeneity()` adapts the DerSimonian-Laird method-of-moments estimator ([DerSimonian and Laird 1986](#)) to the centroid summaries:

$$\begin{aligned}\theta_i &= \frac{S_{m,i}}{S_{m,i} + S_{u,i}}, & Q_i &= S_{wx^2,i} - \frac{S_{m,i}^2}{S_{m,i} + S_{u,i}}, \\ \tau_i^2 &= \max\left(0, \frac{Q_i - \max(N_i - 1, 0)}{\max((S_{m,i} + S_{u,i}) - S_{c^2,i}/(S_{m,i} + S_{u,i}), \varepsilon)}\right).\end{aligned}\tag{2.7}$$

This is a useful heterogeneity diagnostic, but it should be described as a method-of-moments adaptation to stored centroid summaries, not as a full random-effects model fit.

2.6 Continuous Overlap

Once densities have been reconstructed, the code measures shared support through

$$O_i = \int_0^1 \min(\tilde{f}_{1i}(x), \tilde{f}_{2i}(x)) \, dx.\tag{2.8}$$

Numerically, the integral is evaluated on a dense grid after each density is renormalized by trapezoidal integration. This makes the overlap robust to small interpolation drift, but it remains a grid approximation.

2.7 Multiple Testing and p-Value Aggregation

For genome-wide DMP screening, `storey_qvalues()` estimates the null fraction π_0 across a grid of λ values and then computes monotone q-values ([Storey 2003](#)):

$$\begin{aligned}\hat{\pi}_0(\lambda) &= \frac{\#\{p_j > \lambda\}}{(1 - \lambda)m}, & \hat{\pi}_0 &= \min(1, \text{median}_\lambda \hat{\pi}_0(\lambda)), \\ q_{(i)} &= \min_{j \geq i} \frac{\hat{\pi}_0 m p_{(j)}}{j}.\end{aligned}\tag{2.9}$$

The same module also provides Fisher ([Fisher 1925](#)) and Stouffer ([Stouffer et al. 1949](#)) style p-value combination utilities. Those are not central to the detector itself, but downstream packages reuse them conceptually for gene-level aggregation.

2.8 Canonical Biological Effect Size

The repository separates statistical significance from a biological ranking score. The implemented canonical effect size is

$$\text{effect}_i = |\Delta\mu_i| (1 - O_i) \exp\left[-\lambda_{\text{var}} \left(\sqrt{\sigma_{1i}^2} + \sqrt{\sigma_{2i}^2}\right)\right]. \quad (2.10)$$

An optional mean-level weight $g(\bar{\mu}_i)$ can be applied, with the code supporting square-root, linear, saturating, and boundary-threshold variants.

This score is useful operationally, but it is not a test statistic, likelihood ratio, or posterior quantity. It is a deliberately engineered ranking rule. Publication text should therefore describe it as a heuristic biological prioritization score.

2.9 ECDFClassifier

The classifier in `ecdf_classifier.py` turns the centroid histograms into per-class densities and scores a sample $x = (x_1, \dots, x_p)$ via a weighted mean log-density:

$$\text{score}_k(x) = \frac{\sum_{i=1}^p w_i \log f_{ki}(x_i)}{\sum_{i=1}^p w_i}. \quad (2.11)$$

The class probabilities are then computed by temperature-scaled softmax:

$$P(Y = k | x) \propto \exp\left(\frac{\text{score}_k(x)}{T_{\text{eff}}}\right), \quad T_{\text{eff}} = T\sqrt{n_{\text{effective}}}, \quad (2.12)$$

where $n_{\text{effective}} = (\sum_i w_i)^2 / \sum_i w_i^2$ is the inverse-Simpson effective number of weighted loci.

Two stabilizers are important because they materially affect behavior:

1. log-densities are floored before averaging, so extremely small density values cannot dominate the score, and
2. unavailable loci are removed from the denominator by setting their effective weights to zero.

The result is a practical Naive Bayes style classifier ([Domingos and Pazzani 1997](#)), but it is not a fully calibrated generative posterior unless the independence and density assumptions actually hold. Optional Platt scaling ([Platt 1999](#)) may further recalibrate the binary probabilities after the ECDF score has been computed.

2.10 Auxiliary Beta-Based Metrics

`metrics_core.py` still contains exact beta divergences, beta overlap surrogates, and moment calculations. A legacy beta-comparator module (`methyl_utils/comparison.py`) was removed from the active package tree; beta-distance utilities in `metrics_core.py` remain for auxiliary analysis only. They are **not** the main production path for DMP detection plus classification and are not used by the active detector/classifier runtime flow. In publication text, beta distances should therefore be described as an auxiliary branch of the repository, not as the canonical detector-classifier model.

3 MethylCentroid

3.1 Role

`methylcentroid` is the package that turns a cohort of samples into a reusable centroid object. The heavy mathematical lifting is delegated to `methylutils`, but `methylcentroid` fixes the operational semantics of what a centroid means in this repository:

- which samples are aggregated,
- how coverage outliers are handled,
- which sufficient statistics are persisted, and
- how the resulting HDF5 artifacts are organized for downstream steps.

The main implementation lives in `packages/methylcentroid/methyl_centroid/methyl_centroid.py`, with accumulation handled by `methyl_utils/core/centroid_builder.py`.

3.2 What A Centroid Stores

Each persisted centroid contains:

- genomic positions,
- context metadata,
- the sufficient statistics N_i , $S_{x,i}$, $S_{x^2,i}$,
- count summaries $S_{m,i}$, $S_{u,i}$, $S_{c^2,i}$, $S_{wx^2,i}$, and
- per-locus histogram counts used to reconstruct ECDFs.

That is why the centroid is more than a mean profile. It is a compressed empirical-distribution object designed to support both detector-side testing and classifier-side scoring.

3.3 Sample-To-Centroid Update Rule

Given one sample with per-locus methylation fraction

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

the centroid builder updates

$$N_i \leftarrow N_i + 1, \quad S_{x,i} \leftarrow S_{x,i} + x_{si}, \quad S_{x^2,i} \leftarrow S_{x^2,i} + x_{si}^2. \quad (3.1)$$

It also updates the count-derived quantities

$$\begin{aligned}
S_{m,i} &\leftarrow S_{m,i} + m_{si}, & S_{u,i} &\leftarrow S_{u,i} + u_{si}, \\
S_{c^2,i} &\leftarrow S_{c^2,i} + (m_{si} + u_{si})^2, & S_{wx^2,i} &\leftarrow S_{wx^2,i} + \frac{m_{si}^2}{m_{si} + u_{si}},
\end{aligned} \tag{3.2}$$

and increments the appropriate methylation histogram bin for x_{si} .

These updates make the centroid a compact empirical summary rather than a full sample-by-locus matrix.

3.4 Coverage Capping By Binomial Thinning

One of the most important preprocessing choices in this package is optional coverage capping. The code implements binomial thinning:

1. for any locus with coverage $n_i = m_i + u_i > n_{\text{cap}}$,
2. compute $p_i = n_{\text{cap}}/n_i$,
3. draw

$$m'_i \sim \text{Binomial}(m_i, p_i), \quad u'_i \sim \text{Binomial}(u_i, p_i). \tag{3.3}$$

This preserves the methylation fraction in expectation while reducing the influence of ultra-deep loci. In other words, it is a variance-control and robustness device, not a biological model.

The implementation exposes both manual capping and auto-capping heuristics. The thinning itself is principled; the choice of n_{cap} is still a workflow decision.

3.5 Auto-Cap Heuristic

The package documentation describes automatic coverage capping through IQR-style heuristics computed from sampled positions. That should be interpreted as an engineering rule:

$$n_{\text{cap}} \approx Q_3 + k \cdot \text{IQR},$$

with implementation-specific sampling and fallbacks. This is useful operationally, but it is not a statistically identified parameter.

3.6 Why Histograms Matter

The centroid builder stores histogram counts even though the mean and variance are already available. That is because the downstream detector and classifier do not rely only on moments. They reconstruct empirical distributions from the binned counts and then use those reconstructed distributions for overlap calculations, KS testing, and classification.

This makes `methylocentroid` a crucial bridge package: it is the step where sample-level methylation is transformed into an empirical-distribution object that later packages can reuse without reloading the full cohort.

3.7 Publication Guidance

For publication text, the safest description is:

- `methylocentroid` constructs empirical per-locus cohort summaries,
- it preserves both moments and histogram information,
- it optionally reduces extreme coverage through binomial thinning, and
- it is upstream of all ECDF-based detector and classifier logic.

The package should **not** be described as fitting a parametric methylation model. It is a summary constructor for the nonparametric empirical-distribution path.

4 MethylDetector

4.1 Role

`methyldetector` turns aligned centroids into a filtered set of differential methylation positions (DMPs) and exports the classifier artifacts used downstream. The main implementation is `packages/methyldetector/methyl_detector/core/methyldetector.py`.

Conceptually, the detector is a two-stage pipeline:

1. statistical screening of loci, and
2. biological prioritization and export.

That split is important. The code does not rely on a single monolithic significance score.

4.2 Alignment And Candidate Set

The detector starts from aligned centroid pairs, typically one control centroid and one disease centroid for a given chromosome and context. The alignment step reduces both centroids to the common set of genomic positions and propagates the empirical summaries described in [Chapter 2](#) and [Chapter 3](#).

Let $\mathcal{J}$ denote the set of aligned loci under consideration for one comparison.

4.3 Statistical Screening

The detector supports two closely related statistical paths:

- a histogram-based path through `MethylCentroidPair.compare_centroids()`, and
- a more precise ECDF path that recomputes KS statistics directly from reconstructed empirical distributions.

In both cases the active workflow is:

1. compute one p-value p_i per locus $i \in \mathcal{J}$,
2. adjust all p-values to q-values using Storey q-values,
3. keep loci satisfying the configured q-value threshold and any minimum effect thresholds.

At the theory level this is

$$\mathcal{D}_{\text{stat}} = \{i \in \mathcal{I} : q_i \leq \alpha, |\Delta\mu_i| \geq \delta_{\text{min}}\}, \quad (4.1)$$

with the exact choice of α and δ_{min} coming from configuration rather than mathematical derivation.

4.4 Biological Ranking

Once a statistically filtered set exists, the detector ranks loci using the canonical effect score introduced in Eq. 2.10. This gives a second set of priorities that is explicitly biological or practical rather than inferential.

The central design choice is that a locus can be:

- statistically strong but biologically modest,
- biologically large but not significant after multiplicity correction, or
- both.

The detector keeps these notions separate instead of compressing them into one scalar.

4.5 Cumulative Effect Coverage

One of the package’s characteristic filters is cumulative effect-mass selection. After sorting retained loci by decreasing effect size, the detector may keep only the smallest prefix whose cumulative effect mass reaches a specified fraction γ of the total:

$$\sum_{i \in S_\gamma} e_i \geq \gamma \sum_{i \in \mathcal{D}_{\text{stat}}} e_i, \quad (4.2)$$

where e_i is the per-locus effect score.

This is a useful way to compress large DMP sets, but it is a heuristic summarization rule rather than a statistical decision boundary with a formal optimality guarantee.

4.6 Context And Chromosome Granularity

The detector is designed to run at chromosome and context granularity. In practical terms this means:

- each classifier artifact is tied to a specific chromosome,
- contexts such as **CG**, **CHG**, and **CHH** can be modeled separately or together, and
- any final multiclass or multi-chromosome system is assembled later by **methylclassifier**.

That architecture keeps detection local and composable, but it also means that downstream fusion is not part of the detector’s statistical test itself.

4.7 Exported Classifier Artifacts

The detector does more than report DMPs. It also packages the information needed by the ECDF classifier:

- the filtered DMP positions,
- per-class centroid histograms over those positions,
- directions of methylation change, and
- feature weights such as effect size.

Mathematically, this means the detector defines the feature space and the feature weighting scheme for the classifier. The classifier is therefore downstream of detector choices even when its own scoring rule is fixed.

4.8 Discovery versus prediction exports

Interpretation workflows (mapping DMPs to genes, pathway enrichment, comparing stages) benefit from **broader, biologically coherent** DMP lists. Predictive modeling benefits from **smaller, stable** feature sets chosen without conflating biological narrative with classification utility.

The implementation therefore supports two coordinated outputs when `dmp_export_mode=dual` (default in project configs):

- `dmps-{chrom}-discovery.csv`: all biologically filtered loci after importance ranking, **without** the effect-size elbow used to trim the classifier panel (unless `discovery_dynamic_dmp_cutoff_enabled` is turned on).
- `dmps-{chrom}-selected.csv`: the **primary prediction panel** after FeatureCuts balanced-accuracy optimization or the effect-size elbow (`classifier_dmp_selection=featurecuts_validation` when validation samples are configured). Legacy `dmps-{chrom}-classifier.csv` remains for one release cycle.

The saved ECDF classifier pickle always uses the **selected** panel, not the discovery table. MethylMapper and MethylEnricher should read the **discovery** glob (for example `dmps-*-discovery.csv`) in discovery-only modeling modes; FeatureCuts workflows prefer `dmps-*-selected.csv`. Multiclass merge utilities resolve `dmps-*-selected.csv` automatically when present.

For strict evaluation of a deployed classifier, feature selection must be performed **inside** each outer training fold (for example by running detection on fold-specific centroids, as in Monte Carlo train splits); discovery lists built on all samples may still be reported for mechanism, but they must not be the sole basis for performance claims unless that design is stated explicitly.

4.9 Production Freeze Panel

In the production workflow (Chapter 13), the detector can bypass statistical and biological discovery entirely and instead operate from a fixed list of pre-selected loci. This is configured via the `fixed_dmp_panel` parameter in profile `actionConfig.detection`.

When `fixed_dmp_panel` is set to a path pointing at a CSV file that contains at minimum the columns `chromosome` and `position`, the `_run_multi_context` method short-circuits the entire discovery pipeline. No centroids are compared, no KS statistics are computed, and no biological filtering is applied. The columns `context`, `effect_size`, `statistical_dmp`, and `biological_dmp` are inferred or set to unit defaults for any that are missing from the file. The result object is constructed directly from the loaded rows, with `biological_retention_rate = 1.0` and `n_statistical_dmps = n_biological_dmps = n_classifier_dmps` all equal to the CSV row count.

This mechanism serves the production workflow where:

1. DMP positions were already validated across many Monte Carlo splits (stability analysis),
2. only the most reliably recurring positions are retained in the stable panel, and
3. the final model must use exactly those positions on the full dataset without re-running discovery.

The code path is:

```
MethylDetector._run_multi_context()
↓ if config.fixed_dmp_panel is not None
↓ pd.read_csv(fixed_path)
↓ validate columns {"chromosome", "position"} columns
↓ infer missing context / effect_size / flags
↓ export CSV + save classifier model
↓ return MethylDetectorResult (no discovery performed)
```

The exported classifier model (PKL) is identical in structure to one built through normal discovery; downstream tools (`methylassifier`, `methylassifier`) are unaware of whether the positions came from live detection or a fixed panel.

Theoretical note. Using a fixed panel from a previous analysis is a form of transfer. The panel was selected on a different (possibly overlapping) set of samples. The validity of downstream statistical claims depends on whether the panel samples and the inference samples are from the same population. This is an operational design choice, not a statistical guarantee; it should be stated explicitly in any publication that uses the freeze workflow.

4.10 DMP Coverage in New Samples

A frequently encountered diagnostic problem is low DMP coverage in validation or prediction samples: the classifier reports that only 10–30% of DMP positions have usable observations in a given sample. The root cause is almost always a mismatch between coverage thresholds **at centroid construction / DMP discovery** and **at MethylClassifier inference**:

- `min_coverage` in profile `actionConfig.centroid.base_config` and matching `min_coverage` in `actionConfig.detection` — minimum reads per locus when building centroids and when calling DMPs.
- `min_coverage` (or equivalent) in profile `actionConfig.classifier` / `MethylClassifier JSON` — minimum reads required to treat a locus as observed in a *new* sample.

If centroids and detection used `min_coverage = 4` but the classifier requires a higher per-locus depth, loci with 4–9 reads in a test sample are dropped from the feature matrix even though they were part of the DMP panel.

The observable symptom is:

```
sample_dmp_coverage.dmps_used_fraction_median < 0.5
```

in `validation_metrics.json`. Values below 0.3 indicate severe coverage mismatch and nearly always lead to prediction dominated by the majority class.

Recommended practice: Set **MethylClassifier** `min_coverage` (see classifier docs / schema) to match the centroid/detector `min_coverage` used during training (commonly 4). Raise it only when test samples are known to be much deeper than training and locus-level precision is paramount.

4.11 Publication Guidance

The most accurate publication description is:

- **methyldetector** performs nonparametric locus-wise screening,
- applies FDR control through Storey q-values,
- supplements significance with heuristic biological ranking,
- and exports a reduced, weighted empirical-distribution feature set.

When the `fixed_dmp_panel` path is used, the package should be described as applying a pre-specified position set rather than performing novel discovery.

The package should **not** be described as a pure hypothesis-testing engine if the effect-coverage filters are active, because those filters are explicitly post-test prioritization rules.

5 MethylClassifier

5.1 Role

`methylclassifier` is the orchestration layer that applies one or more binary ECDF classifiers to new samples. The mathematical core still lives in `methyl_utils/ecdf_classifier.py`, but `methylclassifier` determines how per-chromosome models, one-vs-rest bundles, and pairwise disease heads are fused into final probabilities.

The relevant implementation files are:

- `packages/methylclassifier/methyl_classifier/core/classifier.py`
- `packages/methylclassifier/methyl_classifier/core/multiclass_ovr.py`

5.2 Binary Classification

For a single binary model, the underlying score is the ECDF log-density average described in Eq. 2.11 and Eq. 2.12, with explicit class priors:

$$\hat{P}(Y = c \mid x) \propto \exp\left(\frac{\log \hat{p}(x \mid c)}{T}\right) \hat{P}(Y = c).$$

From the perspective of `methylclassifier`, the main practical issues are:

- aligning the input sample to the stored DMP order,
- constructing an availability mask for missing loci, and
- optionally applying a stored Platt calibrator.

If the binary classifier is used alone, this is the cleanest statistical story in the package.

5.3 Multi-Chromosome Binary Classification

When a detector run exports one classifier per chromosome, `methylclassifier` computes chromosome-specific binary probabilities and then fuses them using normalized weights:

$$\hat{p}(y \mid x) \propto \sum_{c \in \mathcal{C}} w_c \hat{p}_c(y \mid x_c), \quad \sum_{c \in \mathcal{C}} w_c > 0. \quad (5.1)$$

The code supports several ways to determine the weights w_c :

- fixed configuration weights,
- trimmed mean effect-size weights derived from detector output,
- fitted linear, ridge, lasso, logistic, or elastic-net models trained on validation probabilities.

The last family is best understood as a second-stage meta-model that uses per-chromosome probabilities as features. It can improve prediction, but it is no longer a single coherent generative model.

5.4 One-vs-Rest (OvR) Multiclass Fusion

For multiclass problems the package builds K binary heads and fuses them into a K -class probability vector. The active implementation in `multiclass_ovr.py` uses binary probabilities or logits as intermediate scores and then applies a softmax-like normalization. In symbolic form,

$$\ell_k(x) = \log \hat{p}_k(y = k \mid x) - \log \hat{p}_k(y \neq k \mid x),$$

$$\hat{P}(Y = k \mid x) = \frac{\exp(\ell_k(x))}{\sum_{r=1}^K \exp(\ell_r(x))}. \quad (5.2)$$

This is a practical multiclass reduction. It is not guaranteed to recover a single joint Bayesian posterior unless the binary heads are themselves perfectly compatible. Package metadata now records `ovr_inference_version` and `ovr_fuse_mode` so multiclass semantics are explicit and reproducible.

5.5 Pairwise Control Aggregation

Some project layouts export only pairwise control-vs-disease detectors. In that case the package constructs the control class probability from the geometric mean of the pairwise control probabilities:

$$\hat{p}_{\text{ctrl}}(x) = \exp \left(\frac{1}{K-1} \sum_{k=1}^{K-1} \log \hat{p}_{\text{ctrl}|\text{ctrl-vs-}k}(x) \right). \quad (5.3)$$

The complement $1 - \hat{p}_{\text{ctrl}}(x)$ is then used to form a binary control-vs-rest head before final row normalization.

This is a useful aggregation rule, but it is a heuristic. The geometric mean is not derived in the code from a global multiclass likelihood.

5.6 Bipartite Multi-Control × Multi-Disease Aggregation

The package also supports bipartite layouts with several control strata and several disease strata. Here it aggregates one probability column across several pairwise experts, again using geometric means. The logic is similar to Eq. 5.3 but now applied to either the control column or the disease column depending on which class head is being built.

That construction is convenient for complex cohort hierarchies, but it should be presented as a carefully engineered reduction strategy rather than a textbook probabilistic model.

5.7 Calibration

When enabled, the package can apply Platt scaling (Platt 1999) to the raw binary outputs. In effect, one learns parameters a and b so that a raw score s is mapped to

$$\hat{p}(y = 1 \mid s) = \frac{1}{1 + \exp(as + b)}. \quad (5.4)$$

This improves probability calibration under distribution shift or class imbalance, but it should not be confused with changing the underlying classifier likelihood.

5.8 New Modeling Features in Production

Recent versions of `methylassifier` expose a broader post-detector modeling layer than the original binary ECDF-only story. In practice, production runs may combine several of the following:

- native multiclass histogram model export (`multiclass-classifier.pkl`) from per-comparison detector outputs,
- optional learned multinomial logistic head on top of histogram pre-softmax scores,
- optional isotonic calibration for probability reshaping,
- optional elastic-net stacking for chromosome-level fusion,
- optional dependence-aware block aggregation (`dependence_block_size`, `dependence_block_shrinkage`),
- hierarchical panel readout for family-level reporting.

These are all valid engineering extensions, but conceptually they sit **on top of** the ECDF likelihood core rather than replacing it.

5.8.1 Learned Multiclass Head

The learned head takes base multiclass score vectors $z(x)$ and fits a multinomial logistic layer:

$$\hat{P}(Y = k \mid x) = \frac{\exp(\beta_k^\top z(x) + b_k)}{\sum_{r=1}^K \exp(\beta_r^\top z(x) + b_r)}.$$

This layer is controlled by `multiclass_train_learned_head` and related hyperparameters in profile `actionConfig.detection` (see [reference/configuration-reference](#)). It is especially useful when the raw histogram scores have good ranking behavior but imperfect class boundaries.

5.8.2 Isotonic and Stacking Layers

Two additional modeling switches in profile `actionConfig.classifier` are important for current production behavior:

- `use_isotonic_calibration`: nonparametric monotone calibration of predicted probabilities,
- `use_elasticnet_stacking`: linear meta-model for chromosome-weight fusion under elastic-net regularization.

Both improve empirical calibration or discrimination in many cohorts, but they increase model complexity and should be reported explicitly in methods sections.

5.8.3 Practical Reproducibility

For reproducibility claims, report:

1. whether the learned multiclass head was enabled,
2. whether class balancing was used (`multiclass_learned_class_weight`),
3. whether isotonic calibration and/or elastic-net stacking were enabled,
4. the sklearn version stored in PKL metadata.

Without these details, two runs with the same DMP panel can still produce materially different probabilities.

5.9 Multiclass Learned Head: Class Weighting

When the native multiclass histogram classifier is extended with a learned logistic head (`multiclass_train_learned_head: true`), the logistic regression is fitted on top of the pre-softmax histogram scores from the base model. This second-stage model can accept a `class_weight` parameter (exposed as `multiclass_learned_class_weight` in profile `actionConfig.detection`; see [reference/configuration-reference](#)).

With `multiclass_learned_class_weight: "balanced"`, sklearn computes inverse-frequency weights:

$$w_k = \frac{n_{\text{total}}}{K \cdot n_k}, \quad (5.5)$$

where n_k is the number of training samples in class k , K is the number of classes, and $n_{\text{total}} = \sum_k n_k$. These weights are passed to the loss function of the multinomial logistic regression, penalizing errors on minority classes more heavily.

This is particularly important for the prostate cancer staging problem where the healthy control cohort may be two to four times larger than any individual disease stage cohort. Without balancing, the logistic head learns to predict the healthy class as the default when the pre-softmax histogram scores do not provide strong class separation (for example, when only 20% of DMP positions are covered in a test sample; see § 4.10).

Theoretical note. Class weighting corrects for imbalance in the empirical class frequencies of the training set. It does not correct for population prevalence differences between training and deployment contexts. If the intended clinical population has very different disease prevalence from the training cohort, additional calibration against the target distribution is required.

Reproducibility. Since sklearn 1.4, the version of sklearn used to train the logistic head is saved in the model PKL metadata under the key `sklearn_version`. This allows detection of version mismatches between training and inference environments. The key is written by `_fit_learned_multiclass_head` in `packages/methylclassifier/methyl_classifier/utils/multiclass_bui`

5.10 Missing Data Semantics

A practical but important detail is how the classifier handles unavailable loci. The implementation masks those loci out of the weighted average rather than imputing them with a full modeled distribution. As a result, the effective evidence for a sample depends on how many DMPs are observed for that sample.

This is a sensible engineering choice, but it means that the posterior confidence is partly a function of data availability as well as class separation.

5.11 Publication Guidance

The strongest defensible summary is:

- the binary head is an ECDF-based Naive Bayes style score with explicit priors and optional calibration,
- multi-chromosome fusion is a weighted ensemble,
- multiclass OvR is a reduction from binary heads to a normalized K -class score,
- pairwise control aggregation is heuristic,
- and fitted chromosome weights are second-stage discriminative models, not part of the original density model.

That framing is more accurate than calling the entire package a single Bayesian multiclass classifier.

6 MethylPredictor, MethylValidation, and MethylDiseaseProgression

6.1 Role

`methylpredictor`, `methylvalidation`, and `methylDiseaseProgression` do not introduce a new generative methylation model. Instead they evaluate, operationalize, and synthesize outputs exported by detector/classifier/mapper/enricher stages.

- `methylpredictor` applies trained classifier artifacts to new cohorts and computes prediction summaries.
- `methylvalidation` creates repeated train/validation splits and aggregates run-level metrics across Monte Carlo style repetitions.
- `methylDiseaseProgression` aggregates per-comparison mapper/enricher outputs into ordered cross-stage progression tables and labels.

6.2 MethylPredictor: Labeled Evaluation

When labels are available, `methylpredictor` computes standard classification metrics using `sklearn.metrics`. For n labeled samples with true labels y_i and predictions $\hat{y}_i$,

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\hat{y}_i = y_i\},$$

$$\text{BalancedAccuracy} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k. \quad (6.1)$$

For binary classification the package also reports:

$$\text{Sensitivity} = \text{Recall}_{\text{positive}}, \quad \text{Specificity} = \text{Recall}_{\text{negative}}.$$

For multiclass settings it reports per-class precision, recall, F_1 , as well as macro and weighted F_1 summaries.

These are standard descriptive metrics. Their interpretation depends on how the evaluation cohort was assembled rather than on any special estimator inside the package.

6.3 Blind Prediction Summaries

In blind mode, `methylpredictor` cannot estimate discrimination performance because no truth labels are available. Instead it reports:

- predicted class counts,
- mean predicted probability per class, and
- mean Shannon entropy of the per-sample class probabilities.

For one sample with class probabilities $p_1, \dots, p_K$, entropy is

$$H(p) = - \sum_{k=1}^K p_k \log p_k. \quad (6.2)$$

Low entropy corresponds to confident predictions; high entropy corresponds to diffuse class assignments. This is an uncertainty summary, not an accuracy estimate.

6.4 MethylValidation: Split Strategy

`methylvalidation` implements repeated train/validation splitting by cohort. For each cohort with n samples and train fraction ρ , the code computes

$$n_{\text{train}} = \max(1, \lfloor \rho n \rfloor),$$

and then reduces n_{train} to $n - 1$ if necessary so that the validation set is non-empty for $n > 1$.

This is called “stratified” in the code because each cohort is split separately, but it is not identical to the textbook `StratifiedKFold` abstraction. The package performs independent random shuffles within each cohort and then concatenates the resulting class-specific train and validation groups.

6.5 Monte Carlo Evaluation

The validation package repeatedly:

1. generates train and validation cohorts,
2. writes run-specific project files,
3. runs the full pipeline from centroid through predictor, and
4. aggregates scalar metrics across repetitions.

If m_r denotes some metric from run r , the package reports descriptive summaries such as

$$\bar{m} = \frac{1}{R} \sum_{r=1}^R m_r, \quad s_m^2 = \frac{1}{R-1} \sum_{r=1}^R (m_r - \bar{m})^2,$$

along with empirical percentiles.

This is best interpreted as repeated internal validation of the **whole pipeline**, not as a formal confidence interval for an external deployment population.

6.6 What The Validation Layer Does Not Do

The current implementation does not attempt:

- nested hyperparameter tuning,
- exact sampling-theory confidence intervals for each metric,
- dependence-aware aggregation across repeated splits, or
- external-cohort transportability analysis.

Those omissions are not flaws in themselves, but they matter for publication wording. The package measures empirical variation across repeated pipeline runs; it does not prove generalization bounds.

6.7 Stability Analysis

After completing a set of Monte Carlo runs, the `--stability` mode aggregates DMP recurrence across runs. For each discovery DMP d (a `(chromosome, position)` pair), define the recurrence indicator

$$\mathbf{1}_r(d) = \begin{cases} 1 & \text{if } d \in \mathcal{D}_{\text{disc},r} \\ 0 & \text{otherwise} \end{cases}$$

where $\mathcal{D}_{\text{disc},r}$ is the set of discovery DMPs from run r . When a minimum balanced accuracy threshold $\beta_{\min}$ is configured, only qualifying runs (those with $\widehat{\text{BA}}_r \geq \beta_{\min}$) contribute to the count. Let R^* denote the number of qualifying runs. The estimated recurrence frequency is

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}_r(d). \quad (6.3)$$

A DMP is declared **stable** if $\hat{f}(d) \geq \tau$ for a user-specified threshold τ (default 0.7, `stability_dmp_freq`). The output is a CSV file `stable_dmps_production.csv` containing all stable DMPs with their frequencies, sorted by decreasing frequency.

Gene stability is an analogous measure over enricher gene lists: the frequency at which each gene appears in enrichment results across qualifying runs.

Theoretical note. $\hat{f}(d)$ is an empirical frequency, not a formal probability. It depends on the train fraction ρ , the random seed, and the number of iterations R . With small R the estimate is noisy. The threshold τ is a heuristic engineering choice, not derived from any optimality criterion. Higher τ produces smaller, more reproducible panels; lower τ retains more positions at the cost of including split-dependent noise.

6.8 Production Freeze

The production freeze converts the stable DMP panel into a fixed discovery/interpretable production run on the full dataset. The `--freeze` flag calls `freeze_production_model()` in `packages/methylvalidation/methyl_validation/stability.py`, which:

1. Merges one or more per-chromosome stable DMP CSVs into a single genome-wide panel (`stable_dmps_genomewide.csv`).
2. Loads the base project JSON and injects `fixed_dmp_panel` into resolved `actionConfig.detection` (pointing at the merged CSV).
3. Writes a `production/project.json` with `project_name = "production"` and `output_base` routed to the production directory.
4. Executes `run_pipeline_for_production()`, which runs: `methyl-centroid` → `methyl-detector(fixed panel)` → `methyl-mapper` → `methyl-enricher`.
5. Optionally runs `methyl-disease-progression` when profile `actionConfig.progression.enabled=true`.
6. Writes a `production_summary.json` summarizing the run.

Final model fitting is a separate explicit step: `methyl-validation --model` runs `methyl-classifier` → `methyl-predictor` on `production/project.json`.

If configured (`actionConfig.validation.require_biological_review_for_model=true`), `--model` is blocked until `biological_review_confirmed=true`.

The production project JSON (`production/project.json`) is the artifact that links the frozen model to the `--predictor-only` workflow: it encodes the centroid directories, detection output paths, and classifier paths so that `methyl-predictor` can resolve them without re-running any upstream steps.

6.9 Hybrid Generative Backend (`model_backend=generative_hybrid`)

The `--model` stage now supports a first-class hybrid generative backend in addition to the legacy ECDF path and the tabular sklearn path. This backend is orchestrated by `methylvalidation` and uses the same detector-derived DMP bundle as the other non-ECDF model backends.

6.9.1 Data Contract and Bundle Reuse

The backend does **not** re-derive DMP biology or feature importance. Instead it consumes the `ModelFeatureBundle` generated from detector exports:

- DMP index (`chromosome, position, context`)
- detector-side `effect_size` and `weight` columns
- class labels resolved from project comparisons/groups

This keeps biological weighting aligned with detector outputs and avoids backend-specific feature redefinition.

6.9.2 Modeling Structure

The implemented hybrid backend follows a VAE-style latent-density pattern:

1. encode weighted methylation features into a latent representation z (current implementation uses a linear latent encoder surrogate),
2. fit class-conditional latent densities (currently diagonal Gaussian),
3. compute class posteriors via Bayes-style normalization.

In symbolic form, for classes $k = 1, \dots, K$:

$$p(z \mid y = k) \approx \mathcal{N}(\mu_k, \text{diag}(\sigma_k^2)),$$
$$\hat{P}(y = k \mid z) = \frac{\pi_k p(z \mid y = k)}{\sum_{r=1}^K \pi_r p(z \mid y = r)}.$$

This yields likelihood-based scoring for both binary and multiclass settings while preserving the detector-derived feature index.

6.9.3 Orchestration in `--model`

When profile `actionConfig.validation.model_backend="generative_hybrid"`, `run_pipeline_for_model()` executes:

1. **model-bundle**: `build_model_feature_bundle(...)`
2. **generative-train**: `train_generative_model(...)`
3. **generative-predictor**: `predict_generative_model_from_project(...)`

instead of the legacy `methyl-classifier -> methyl-predictor` ECDF chain.

6.9.4 Configuration Surface

The backend is controlled under profile `actionConfig.validation`:

- `model_backend`: "generative_hybrid"
- `model_bundle_dir` (and legacy `model_weight_column`; weighting is canonicalized to `effect_size`)
- `generative_latent_dim`
- `generative_kl_weight`
- `generative_density_type` (currently `diag_gaussian`)
- `generative_epochs`, `generative_batch_size`, `generative_seed`
- `generative_calibrate`
- `covariates_path`, `covariate_id_column`
- `generative_covariates_strict`

Those fields are part of the same validation config object as Monte Carlo settings, so backend choice is fully reproducible from the merged profile and `mc_config.json` snapshot.

6.10 MethylDiseaseProgression Synthesis

`methyl-disease-progression` is a post-freeze synthesis step that treats disease groups as an ordered stage sequence and aggregates entity-level signals across stages.

6.10.1 Inputs and Stage Resolution

For each ordered stage, the code reads:

- mapper combined gene table (`all-gene_name-combined.csv` by default),
- enricher pathway table (`enrichment_merged.csv`, fallback `enrichment_top_q0.05.csv`),
- optional module table (`modules_ranked.csv`).

Stage order comes from:

1. CLI `--ordered-comparison-labels` (highest precedence),
2. profile `actionConfig.progression.ordered_comparison_labels` (or alias `ordered_disease_groups`),
3. project comparison order.

6.10.2 Entity Scores

The synthesis creates long-format tables for genes, pathways, and modules with (`entity_type`, `entity_id`, `stage_index`, `rank`, `score`).

- **Genes:** score is max of available gene-weight columns per gene (e.g. `total_weight`, `gene_importance`).
- **Pathways:** when q-values exist, score is transformed to

$$\text{score}_{\text{pathway}} = -\log_{10}(\max(q, 10^{-300})),$$

so larger values indicate stronger enrichment. - **Modules:** score is the module score exported by `modules_ranked.csv`.

6.10.3 Progression Labels

For each entity across ordered stages, the package assigns categorical progression labels:

- `stage_specific`, `early_only`, `late_only`,
- `stable_across_stages`,
- `monotonic_up` / `monotonic_down` (based on adjacent score differences over at least three points).

These are deterministic rule-based labels, not fitted trajectory models.

6.10.4 Outputs

The progression step writes:

- genes_long.csv
- pathways_long.csv
- modules_long.csv
- entities_progression_labels.csv
- summary.json
- optional report.md (when report_md=true)

This layer is best interpreted as structured biological summarization across stages, not causal disease dynamics inference.

6.10.5 Optional gene set overlap metrics

When metrics are enabled and a profile is provided, the tool writes per-stage overlap tables as CSV plus a JSON summary for plotting. **Preferred config (v1 plan):** nested profile `actionConfig.progression.gene_set_metrics` with `enabled`, optional `gene_universe` (`mapper_all`, `mapper_top_n`, `mapper_min_weight_quantile`), optional `output_basename` (default `stage_gene_set_fractions` when this block is used), plus `gene_set_profile` (path string, `{"path": ...}`, or inline categories with `{id, label, genes}`) or `disease_context` (e.g. `prostate_cancer` → `bundled profiles/prostate_cancer.json`). **Legacy flat keys** remain supported: `gene_set_metrics_enabled`, `gene_sets_path`, `disease_profile`, and `gene_set_denominator / gene_set_top_n / gene_set_weight_quantile` (default CSV basename `stage_gene_set_metrics`). For each stage and category, outputs include `n_universe` (mapper genes after optional filtering), `n_overlap`, and `fraction = n_overlap / n_universe`. These fractions are descriptive checks against curated hypotheses; they complement pathway q-values and do not replace enrichment statistics or imply causal stage biology.

6.10.6 Stage-wise enrichment interpretation (prostate cancer template)

For prostate cancer stage series (PCa1 → PCa4), a common interpretation template is:

Stage	Prostate-specific signal	NKX3-1 lineage TF	Mitochondrial/OX-PHOS	Overall pattern	Interpretation
PCa1 (early)	very strong	high	low/secondary	clean prostate identity	early prostate cancer phenotype with strong lineage specificity

Stage	Prostate-specific signal	NKX3-1 lineage TF	Mitochondrial/OX-PHOS	Overall pattern	Interpretation
PCa2	strong	moderate down	moderate up	mixed	transition phase: partial lineage erosion with metabolic emergence
PCa3	moderate	low down-down	high up-up	shifted	inflection point: lineage programs weaken while metabolic programs dominate
PCa4 (late)	moderate (still present)	weak	very strong	metabolic-dominant	advanced phenotype with strong mitochondrial/OX-PHOS signature and diluted lineage identity

This stage-wise pattern is exactly why mapper -> enricher -> progression is valuable: it tests whether selected DMP/gene signals produce coherent biological trajectories rather than isolated statistical hits.

6.10.7 Key axes of progression (what is changing)

Feature	Direction across stages	Meaning
Prostate identity (NKX3-1, prostate terms)	decreasing	Loss of lineage control and differentiation state
Disease specificity (prostate cancer terms)	slight decrease but stable	Disease signal persists across all stages
Mitochondrial respiration (Complex I, OXPHOS)	strongly increasing	Metabolic rewiring becomes dominant in later stages
Pathway structure	simple -> complex	Transition from cleaner single-axis signal to multi-system adaptation

6.11 Two Workflows Summary

MethylValidation supports two primary operational modes, described in full in Chapter 13. A brief summary for theory readers:

Workflow 1 — Model Creation. Repeated Monte Carlo runs with `--stability` estimate DMP recurrence frequencies from centroid+detector iterations. The `--freeze` step then builds production biological outputs on the stable panel (`fixed_dmp_panel`) and may run progression synthesis. The `--model` step then performs final classifier/predictor execution after optional biological gate checks.

Workflow 2 — Model Use for Prediction. The `--predictor-only` flag evaluates the frozen production model on repeated random holdouts without retraining. This workflow answers the question: *what is the empirical distribution of classification performance for the frozen production classifier under random sampling from the available cohort?*

The two workflows share the same split-and-aggregate infrastructure (`split.py`, `project_gen.py`, `pipeline_runner.py`) but differ in which pipeline steps execute per iteration. Workflow 1 iterations run centroid+detector; Workflow 2 iterations run predictor only.

6.12 Publication Guidance

For publication text:

- describe `methylpredictor` as the evaluation and application layer for trained classifiers,
- reserve discrimination claims for labeled validation cohorts,
- treat blind-mode entropy as an uncertainty diagnostic only,
- describe `methylvalidation` as repeated split-sample internal validation of the end-to-end workflow,
- report the stability threshold τ and the number of qualifying runs R^* when using `--stability`, and
- state explicitly whether the production model was trained on all samples or on a split-specific subset.

That is both accurate to the code and defensible statistically.

7 MethylMapper

7.1 Role

`methylmapper` converts locus-level detector output into gene- and region-level annotations. It has two distinct mathematical layers:

1. deterministic genomic mapping through bedtools-style interval logic or an external Azure SQL stored procedure, and
2. gene-level aggregation and disease-evidence enrichment.

The relevant code lives primarily in:

- `packages/methylmapper/methyl_mapper/bedtools_mapper.py`
- `packages/methylmapper/methyl_mapper/mapper.py`
- `packages/methylmapper/methyl_mapper/gene_disease_enricher.py`

7.2 Interval Mapping

The bedtools path maps DMP coordinates to genomic features such as promoters, exons, introns, and gene bodies. This part is deterministic interval algebra rather than statistical inference. Feature-level weights may then be attached, for example promoter weights or exon weights, but those weights are biological priors chosen by the workflow rather than estimated model parameters.

The Azure SQL path calls a stored procedure (`spMapDMP2Genes`). Because the stored procedure logic is external to the repository, its exact mapping semantics should be documented as an external dependency rather than as a transparent in-repo algorithm.

7.3 Gene-Level Combined p-Values

For genes with several mapped DMPs, the package computes a signed weighted Stouffer aggregation. Given DMP p-values p_j , weights w_j , and direction signs $s_j \in \{-1, +1\}$,

$$z_j = \Phi^{-1}(1 - p_j/2), \quad \tilde{z}_j = s_j z_j,$$
$$Z_{\text{gene}} = \frac{\sum_j w_j \tilde{z}_j}{\sqrt{\sum_j w_j^2}}, \quad p_{\text{gene}} = 2 [1 - \Phi(|Z_{\text{gene}}|)] . \quad (7.1)$$

The package then applies Storey q-values to the resulting gene-level p-values.

This is a standard meta-analytic construction in form (Stouffer et al. 1949), but its interpretation requires care: mapped DMPs within one gene are correlated along the genome, so the independence assumption behind classical Stouffer aggregation is only approximate.

7.4 Biological Importance Score

Gene- and feature-level biological importance aggregates detector `effect_size` with a configurable **biology-weight matrix** (feature $\times$ hyper/hypo). Per mapped DMP:

$$w_i = f_i \cdot b(f_i, s_i), \quad |t_i| = w_i \cdot |e_i|, \quad \tilde{t}_i = \text{sign}(\Delta\mu_i) |t_i|$$

where f_i is recurrence **frequency**, e_i is detector **effect_size**, b is the biology weight for feature bucket f_i and direction s_i .

Gene-level rank:

$$I_{\text{gene}} = \left(\sum_i |t_i| \right) \cdot \frac{|\sum_i \tilde{t}_i|}{\sum_i |t_i|} \cdot \sqrt{\bar{f}}$$

Feature-level columns `feature_importance_{promoter,exon,...}` use the same pattern within each bucket. Full defaults and export columns are documented in `packages/methylmapper/docs/BIOLOGICAL_IMPORTANCE`.

Downstream packages sort and weight genes by `gene_importance`, not row-averaged legacy summaries.

7.5 Stable Gene Set Optimization

The package also contains routines that choose the number of top DMPs required to obtain a stable gene set. These routines are operationally useful, but they are not estimators with standard sampling distributions. Their parameters, step sizes, and stability thresholds should therefore be documented as tuning knobs.

7.6 External Evidence Enrichment

`GeneDiseaseEnricher` queries external services such as DisGeNET (Piñero et al. 2020), Open Targets (Ochoa et al. 2023), and optional Grok-mediated evidence pipelines. This is a valuable biological annotation layer, but it is not in-repo statistical inference. The final evidence depends on:

- service availability,
- service version,
- cached responses,

- threshold profiles such as `strict` or `balanced`, and
- optional LLM-assisted synthesis.

For publication, that layer must be framed as external evidence gathering and summarization rather than as a mathematically identified part of the detector.

7.7 Publication Guidance

The most defensible description is:

- interval mapping is deterministic annotation,
- gene-level p-values are weighted Stouffer aggregates with dependence caveats,
- gene importance is heuristic,
- and disease evidence is external-service-backed annotation.

That is still scientifically useful, but it is not the same kind of object as the upstream ECDF hypothesis tests.

7.8 Mapper As Biological Validation Layer

Although `methyMapper` is downstream from DMP detection, it is a key validation checkpoint for selection quality. The practical question is whether selected loci project to coherent gene-level biology:

- Do top genes retain known lineage markers?
- Do mapped signals remain stable across runs and cohorts?
- Do gene-level aggregates bridge naturally into enrichment/module themes?

In this sense, mapper outputs are not only annotations; they are a sanity check that locus-level selection carries biologically meaningful structure into gene space.

8 MethylEnricher

8.1 Role

`methylenricher` is a downstream interpretation layer. It takes mapped genes or weighted gene lists and organizes them into pathway-level and module-level summaries. The package mixes three kinds of methodology:

- external enrichment analysis through Enrichr ([Chen et al. 2013](#)),
- graph-based pathway grouping,
- optional PPI topology refinement,
- optional interactive Cytoscape-style exploration (Dash),
- heuristic module ranking and disease-prior labeling.

Because of that mixture, this package should be described as a discovery and prioritization layer, not as a single statistical model.

8.2 Over-Representation Analysis

The enrichment step is delegated to Enrichr through `gseapy.enrichr`. In practice this means the repository submits a gene list to an external enrichment engine and then reads back adjusted p-values, odds ratios, and gene memberships for the selected libraries.

The statistical test itself therefore lives outside the repository. The in-repo code is responsible for:

- choosing the input genes,
- selecting the enrichment libraries,
- filtering the returned tables, and
- carrying forward the adjusted p-values into later scoring steps.

Publication text should cite Enrichr ([Chen et al. 2013](#)) and record the exact library set used in the analysis. In MethylEnricher this can be done either by logging an explicit `libraries` list or by logging a named `library_preset` (for example `cancer-extended` for cancer-focused studies).

8.2.1 Library sets used by MethylEnricher

The package now supports both a standard default set and named cancer presets.

Standard default (DEFAULT_LIBRARIES)

- KEGG_2021_Human
- Reactome_2022

- GO_Biological_Process_2023
- GO_Molecular_Function_2023
- GO_Cellular_Component_2023
- MSigDB_Hallmark_2020
- WikiPathway_2023_Human

cancer-core preset

Standard default plus:

- ChEA_2022
- ENCODE_and_ChEA_Consensus_TFs_from_ChIP-X
- TRRUST_Transcription_Factors_2019
- DisGeNET
- Jensen_DISEASES
- GWAS_Catalog_2019

cancer-extended preset

cancer-core plus:

- DSigDB
- DGIdb_Drug_Targets_2024
- LINCS_L1000_Chem_Pert_up
- LINCS_L1000_Chem_Pert_down
- miRTarBase_2017

When both `libraries` and `library_preset` are provided, explicit `libraries` take precedence.

8.3 Pathway Graph Construction

The package builds a pathway graph in which nodes are pathways and edges represent gene-set overlap. In the simplest case the edge weight is based on Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (8.1)$$

or a closely related overlap score. Community detection is then performed with the Louvain method ([Blondel et al. 2008](#)), with connected-components fallbacks if the required package is unavailable.

This is a standard graph heuristic for organizing pathways, but the resulting modules depend materially on the chosen threshold, graph construction rule, and Louvain resolution behavior.

When broad library presets are used (for example **cancer-extended**), module construction can include tens of thousands of enrichment terms. The implementation now supports pre-clustering term reduction so modules can be built from a focused subset:

- q-value gate (`module_cluster_max_q`): keep only terms with adjusted p-value below a threshold,

- per-library cap (`module_cluster_top_terms_per_library`): keep top N terms per library before graph construction.

These controls change the module graph itself, not only the display layer, so they should be reported in methods sections.

8.4 Why This Layer Matters For DMP Validation

`methylnapper` and `methylenricher` are not upstream feature selectors, but they are the main biological validation layer for the selected DMP panel. In practice, if the detector-selected loci are biologically coherent, downstream enrichment should recover:

- lineage-consistent terms (for prostate cancer: prostate epithelial identity programs),
- stage-consistent pathway transitions (for example increasing mitochondrial/OXPHOS emphasis in later disease),
- reproducible themes across mapping, enrichment, and module summaries.

This is not formal proof of causal correctness, but it is a strong orthogonal consistency check on DMP/gene selection quality.

8.5 Module Enrichment Score

`compute_module_enrichment_score()` converts a module into a scalar ranking score by combining:

- mean $-\log_{10}(q)$ across pathways,
- number of pathways,
- number of genes,
- mean gene weight.

In normalized form the score is

$$\text{score}_{\text{enrich}} = 0.4 \tilde{m}_{-\log_{10} q} + 0.2 \tilde{n}_{\text{pathways}} + 0.2 \tilde{n}_{\text{genes}} + 0.2 \tilde{m}_{\text{gene-weight}}, \quad (8.2)$$

where each tilded quantity is clipped to $[0, 1]$ after dividing by a hand-chosen scale such as 10, 50, or 200.

This is explicitly heuristic. It is a ranking function, not an enrichment test.

8.6 Disease Relevance Prior

The package also computes a disease-relevance score by overlap with a curated gene set:

$$\text{score}_{\text{disease}} = \frac{|G_{\text{module}} \cap G_{\text{disease}}|}{|G_{\text{module}}|}. \quad (8.3)$$

When no disease set is available the package returns a neutral value of 0.5. By default the code ships with a prostate-cancer-oriented prior gene set, so that default must not be described as disease-agnostic.

The final module score is then

$$\text{score}_{\text{final}} = 0.7 \text{ score}_{\text{enrich}} + 0.3 \text{ score}_{\text{disease}}. \quad (8.4)$$

This is again a heuristic prioritization rule.

8.7 Optional PPI Network Refinement

When enabled (`network_refinement.enabled=true` or matching CLI flags), the package adds a second graph layer from gene-level protein-protein interactions (STRING API or a local edge list). For each module it computes:

- induced-subgraph density,
- mean of a **per-gene node metric** on the subgraph (by default the signal-weighted combined hub score, not only raw degree centrality),
- largest connected-component ratio.

Raw PPI hubs are often dominated by high-database-degree “infrastructure” genes. By default, node metrics are combined with **input methylation weights** (e.g. `mean_effect_size` / `gene_importance` from the mapper CSV): normalized centrality terms are blended into `topology_score`, then multiplied by min-max normalized weights and an optional disease-prior boost to form `combined_hub_score`. Hub lists and module coherence use this score unless `hub_ranking_mode=topology`.

These features are combined into a heuristic PPI coherence score:

$$\text{score}_{\text{ppi}} = 0.4 \text{ density} + 0.3 \text{ mean_node_metric} + 0.3 \text{ largest_component_ratio}. \quad (8.5)$$

The final ranking can then be blended with weight $w \in [0, 1]$:

$$\text{score}_{\text{blended}} = (1 - w) \text{ score}_{\text{final}} + w \text{ score}_{\text{ppi}}. \quad (8.6)$$

This extension is still heuristic. It should be reported as structural corroboration of module coherence, not as formal inference or disease-association evidence.

8.8 Visualization Modes

The package supports both reproducible offline artifacts and interactive inspection:

- offline: Plotly HTML and Cytoscape.js HTML+JSON exports,
- interactive: optional `dash_cytoscape` viewer (`network_plot=dash`) with module and edge-threshold filtering.

For publication and auditability, offline artifacts should remain the canonical archived output; Dash should be treated as an exploratory interface.

8.9 Project-Orchestrated Execution

In pipeline runs, MethylEnricher is commonly launched with `--project` and reads merged `actionConfig.enricher`. When the project contains multiple comparison groups, the runtime can execute one enrichment/module pass per group using per-comparison mapper inputs and enricher outputs. If explicit input/output arguments are passed, execution falls back to a single run.

Configuration precedence for libraries is:

1. explicit `libraries`,
2. `library_preset`,
3. built-in defaults.

This precedence should be reported in methods text to ensure reproducibility of enrichment calls.

8.10 Theme Normalization And Labels

The package includes theme normalization based on substring matching against a JSON mapping file, plus discrete labels such as **High**, **Medium**, and **Low** relevance. Those labels are workflow conveniences. They should not be presented as probabilistic classifications.

8.11 Publication Guidance

The most accurate description is:

- enrichment significance comes from an external ORA engine,
- pathway modules come from graph clustering over pathway overlap,
- optional PPI refinement comes from STRING or local edge lists with explicit score thresholds and community settings,
- module ranking is heuristic,
- and disease relevance is a curated prior overlap rather than a learned posterior.

That combination can be highly useful for interpretation, but it should be framed as an annotation and prioritization pipeline rather than as formal inference.

9 MethylAlignmentQC

9.1 Role

`methylalignmentqc` parses alignment and duplication metrics from **Picard-style deduplication files** and **Parabricks WGBS JSON**, then writes normalized per-sample JSON with **guardrails** that gate SamplePrep before methylation extraction.

Operator guide: [Usage ch.03 — Sample Prep and QC](#).

The package is deterministic: parsing, normalization, threshold checks, and cycle-quality screening — not new statistical inference on methylation.

9.2 Input metrics

9.2.1 Picard-style deduplication metrics

Parabricks emits tab-separated `*deduplicate_metrics.txt` (or `*duplication_metrics.txt`). The parser (`methyl_alignment_qc/core/parser.py`) reads columns such as `PERCENT_DUPLICATION`, read-pair counts, and unmapped reads. These populate `summary_stats` on the export JSON.

Optional config guardrails (`actionConfig.alignment_qc.optional_guardrails.duplication_rate_max`, `min_pf_reads`) can fail samples with excessive PCR duplication even when cycle quality passes.

9.2.2 Parabricks WGBS metrics JSON

Each sample directory should contain `{sample_id}.json` with sections including:

- `quality_yield` — PF reads, Q30 bases
- `mean_quality_by_cycle` — per-cycle mean Phred (Read 1 then Read 2)
- `gc_bias_summary`, `insert_size_metrics`
- `pre_adapter_summaries` — deamination and OxoG artifact qscores

Guardrails in `wgbs_parabricks_qc.py` evaluate PF%, Q30, mean/min cycle quality, GC/AT dropout, median insert size, and deamination/OxoG scores before extraction proceeds.

9.2.3 Alignment-layer guardrails

Beyond sequencing/library checks, `alignment_derived_qc.py` derives **mapping rate**, **secondary/supplementary rate**, and **GC coverage uniformity** from Picard dedup and GC bias tables. When `alignment_guardrails.flagstat_enabled` is true, `bam_flagstat.py` runs `samtools flagstat` on the aligned BAM and gates **properly paired rate** and supplementary alignments.

Analyte profiles enable these guardrails by default for `cfdna` and `buffy_coat`. See the [sample preparation flow](#) for thresholds and operator guidance.

9.3 Export JSON (V2)

The worker writes row-oriented JSON validated against `schemas/config/alignment_qc/exported_sample_qc_v2`. Key blocks:

Block	Role
<code>guardrails.details</code>	Per-metric pass/fail, value, normal range, operator message
<code>guardrails.overall_pass</code>	Bound to workflow <code>qcPass</code>
<code>guardrails.screening</code>	Cycle-quality disposition and recommended trim bases
<code>qc_history</code>	Append-only audit (initial + post-remediation retries)
<code>alignment_stats</code>	Derived mapping and GC uniformity metrics
<code>alignment_flagstat</code>	<code>samtools flagstat</code> summary (when enabled)

9.4 Cycle screening and remediation

When `cycle_screening.enabled` is true, `cycle_quality_screening.py` inspects Read 1 and Read 2 separately. Dispositions include:

Disposition	Meaning
<code>USE_CURRENT_ALIGNMENT</code>	Pass
<code>REALIGN_TRIM</code>	Fixable read-end dip; fastp trim + forced realign
<code>INVESTIGATE_MULTI_REGION</code>	Manual review
<code>INVESTIGATE_GUARDRAIL_ONLY</code>	Cycles OK; check duplication/depth
<code>NOT_FIXABLE</code>	Terminal fail

SamplePrep binds `remediateAlignment` from screening output; FASTQs must remain on disk until final QC disposition.

9.5 cfDNA fragmentomics (optional)

When `primary_analyte` is `cfdna`, insert-size histogram guardrails may run via `fragmentomics.py` after alignment QC passes and before extraction.

9.6 Summary statistics

When the package computes duplication rate, it applies straightforward algebra:

$$\text{duplication rate} = \frac{\text{duplicate reads}}{\text{total reads}}. \quad (9.1)$$

9.7 Publication guidance

Describe `methylationqc` as:

- a QC ingestion and normalization layer,
- dependent on upstream Picard/Parabricks metrics,
- deterministic in parsing and guardrail logic, and
- gating extraction via SamplePrep workflow bindings.

9.8 Related

- Package usage: `packages/methylationqc/docs/USAGE.md`
- Implementation guide (remediation, extraction filtering, cfDNA fragmentomics): [Sample preparation flow](#)
- Extraction QC (downstream gate): Chapter [10](#)

10 MethylExtractionQC

10.1 Role

`methylextractionqc` evaluates **MethylExtractor output** after GPU methylation extraction. Alignment QC confirms the BAM; extraction QC confirms the methylation call layer has sufficient coverage and conversion-quality signal before HDF5 files enter downstream cohort analysis.

Operator guide: [Usage ch.03 — Sample Prep and QC](#).

10.2 Input: extraction manifest

MethylExtractor writes `{sample_id}.extraction_manifest.json` with:

- **summary** — genome-wide CpG weighted mean coverage, CHH/CHG methylation levels
- **per_chromosome** — per-chromosome `CG.mean_coverage` and related stats
- **read_filtering** — aggregated read filter stats (`reads_seen`, `reads_used`, `read_retention_rate`, per-reason drop counts)

10.3 Guardrails

`methylextractionqc/guardrails.py` evaluates Pass/Fail checks (defaults):

Guardrail	Default	Interpretation
<code>cpg_weighted_mean_coverage</code>	10×	Sufficient read depth at CpG sites
<code>chh_methylation_level</code>	0.02	Conversion sanity (when CHH extracted)
<code>chg_methylation_level</code>	0.02	Same for CHG
<code>chromosome_completeness</code>	all expected chromosomes	Every project chromosome present with CG block
<code>chromosome_uniformity</code>	autosomal min/median 0.5	No severe localized dropout
<code>read_discard_fraction</code>	0.9	Catastrophic read loss (<code>1 - read_retention_rate</code>) — wrong reference, contamination, or mis-set filters

The `read_discard_fraction` check reads the manifest `read_filtering` block; when that block is absent (manifests predating read-filtering stats) the check is skipped as a pass so older samples are not retroactively failed.

`guardrails.overall_pass` maps to workflow `extractionQcPass`. Failures are terminal in SamplePrep today (no automatic retry).

Output: `{sample_id}.extraction_qc.json` (schema: `schemas/extraction_qc_output.schema.json`).

10.4 Configuration

Override thresholds (including `max_discard_fraction`) via profile/site `actionConfig.extraction_qc.guardrail`.

10.5 Why both alignment and extraction QC

Question	Answered by
Did sequencing/alignment produce a usable BAM?	Alignment QC (<code>methyl-qc</code>)
Did extraction yield enough confident methylation signal?	Extraction QC (<code>methyl-extraction-qc</code>)

A sample may pass alignment but fail extraction (coverage collapse). Conversely, remediation after alignment QC avoids GPU extraction on BAMs that will not support stable statistics.

10.6 Publication guidance

Describe extraction QC as deterministic threshold checks on exporter-reported coverage and non-CpG methylation fractions — not a new methylation estimator.

10.7 Related

- Package usage: `packages/methylextractionqc/docs/USAGE.md`
- Alignment QC (upstream gate): [Chapter 9](#)

11 Limitations and Open Questions

11.1 Why This Chapter Exists

The repository is scientifically useful, but it is not served well by a perfectly tidy story. The code mixes exact formulas, approximations, heuristics, and external-service calls. This chapter records the main caveats that should be understood before publishing claims based on the pipeline.

11.2 The “ECDF-Only Pipeline” Claim Is Too Broad

The centroid-detector-classifier path is indeed ECDF-centered. However:

- `methylnmapper` uses weighted Stouffer aggregation and heuristic importance scores,
- `methylnricher` uses external enrichment plus heuristic module scoring,
- `methylnalignmentqc` is deterministic QC parsing, not ECDF inference.

Therefore the monorepo as a whole should not be marketed as purely ECDF-only. The safer statement is that the **core supervised detection and classification path** is ECDF-centered.

11.3 Dependence Is Real

Several methods rely on assumptions that are standard but imperfect for methylation data:

- loci within one chromosome are correlated,
- mapped DMPs within one gene are correlated,
- binary OvR heads are not guaranteed to be jointly compatible,
- repeated validation splits are not independent external studies.

This does not invalidate the workflow, but it does change how literally one should interpret p-values, q-values, and meta-analytic combinations.

11.4 Approximation Layers

The code intentionally trades exact raw-data procedures for scalable summaries:

- KS statistics are evaluated on a grid and calibrated asymptotically,
- Mann-Whitney is reconstructed from histograms rather than raw ranks,
- ECDF overlap is integrated numerically after spline reconstruction,
- some distance calculations average only over pairwise-overlapping loci.

These approximations should be reported as approximations, not hidden behind textbook method names alone.

11.5 Heuristic Layers

The following components are best described as engineering decisions:

- the canonical effect-size ranking,
- cumulative effect-mass selection,
- geometric-mean control aggregation for pairwise OvR bundles,
- module scoring constants in `methylenricher`,
- stability thresholds for choosing top DMP counts in `methyldmapper`,
- auto-cap rules for coverage trimming.

These are often useful, but their outputs are not calibrated probabilities or test statistics.

11.6 External Dependencies

Some results depend on systems whose internal logic is not fully versioned inside this repository:

- Enrichr,
- DisGeNET,
- Open Targets,
- Azure SQL stored procedures,
- optional Grok/xAI-assisted evidence synthesis.

Any publication using those layers should record service versions, access dates, and cache policies. Otherwise the results are reproducible only in a weak operational sense.

11.7 Legacy Or Experimental Remnants

The repository still contains legacy components, but their status is now clearer:

- The removed `packages/methylutils/comparison.py` beta-comparator path is **not** used by the active detector/classifier production flow (see §2.10 in Chapter 1).
- A few package-local docs can lag behind code evolution between releases.

To avoid methodological confusion, interpret these as **historical/auxiliary artifacts**, not active runtime estimators.

11.7.1 Current safeguards

The current docs/code governance already reduces this risk:

1. The Quarto theory book (`docs/theory`) is the canonical method reference.
2. Production workflow chapters describe the active ECDF-first detector/classifier path and the current model backends explicitly.
3. Legacy modules remain in-repo for compatibility and experimentation, but publication claims should cite only methods documented as active in this book.

11.7.2 Deprecation Status Snapshot

Component / path	Status	Practical guidance
methylutils ECDF core (<code>core/distribution_views.py</code> , <code>ecdf_classifier.py</code> , centroid pair ECDF comparison path)	active	Safe to cite as current detector/classifier mathematical backbone.
methylvalidation model backends (<code>ecdf</code> , <code>tabular_sklearn</code> , <code>generative_hybrid</code>)	active	Safe to cite when backend and key hyperparameters are reported.
methylmapper, methylenricher, methyldiseaseprogression workflow synthesis	active	Safe to cite as downstream interpretation/synthesis layers (with heuristic/external-service caveats).
<code>methyl_utils/comparison.py</code> beta-comparator module	removed	Historical; do not describe as active production detector path.
Incomplete/retired experimental branches (historical clustering and partial DP-like ideas)	experimental	Do not cite as validated production methodology.

11.8 Publication-Ready Positioning

The strongest defensible manuscript framing is:

1. the core pipeline summarizes methylation into centroids with empirical distributions,
2. it detects DMPs with nonparametric, ECDF-based screening and FDR adjustment,
3. it classifies samples with weighted empirical-distribution likelihoods,
4. and it layers biological annotation and pathway-level interpretation on top.

Anything stronger than that risks overstating what the code actually does.

11.9 What Would Strengthen The Theory Further

The current implementation could support stronger future papers if the repository adds:

- finite-sample benchmarking of histogram-based Mann-Whitney against raw-rank Mann-Whitney,
- calibration studies for the effective-temperature ECDF classifier,
- dependence-aware gene aggregation,
- more explicit modeling of multiclass consistency,
- and clearer retirement or isolation of legacy experimental branches.

Those are natural next steps, but they are not yet the state of the codebase.

Part II

Part II: Configuration Semantics and Workflow Theory

12 The Project Configuration File

12.1 Role and design principle

MethylPipeline separates **study facts** (cohorts, paths, comparisons) from **tool parameters** (detector thresholds, MC iteration counts, mapper windows). The study manifest — usually `project_<name>.json` on `/work/projects/<study>/configs/` — holds only study-level fields validated by `ProjectConfig` in `packages/methylutils/methyl_utils/pipeline_config.py`.

Study manifests must not contain `step_config`. Tool parameters live in profile and site `actionConfig`, resolved at workflow plan time and worker claim time into `resolvedConfig`.

12.2 Four-layer configuration

Layer	Artifact	Owns
Study manifest	<code>project_*.json</code>	Cohorts, comparisons, paths, regulatory , validation_partitions
Pipeline profile	<code>*.profile.json</code>	actionConfig parameter packs + scope booleans
Site manifest	<code>/work/site/methyl_site.json</code>	Genomes, GTF, caches, deployment-wide defaults
DomainProgram	<code>*.program.json</code>	Control flow; optional per-action with / stepOverride

Precedence (highest wins): program/instance override → profile `actionConfig` → analyte defaults → site `actionConfig` → *(no Python fallback for tunable science knobs)*.

See Chapter 13 for how Monte Carlo and production workflows consume these layers.

12.3 Top-level study manifest fields

Field	Type	Description
<code>project_name</code>	string	Output directory name under <code>output_base</code> .
<code>laboratory</code>	string	Metadata; appears in centroid H5 attributes.
<code>batch</code>	string	Metadata; appears in centroid H5 attributes.
<code>output_base</code>	string (path)	Root directory for all pipeline outputs.
<code>samples_base_path</code>	string (path)	Base directory for resolving relative sample paths in cohort CSVs.
<code>chromosomes</code>	list of strings	Chromosomes to process (e.g. ["1", ..., "22", "X", "Y"]).
<code>contexts</code>	list of strings	Methylation contexts: "CG", "CHG", "CHH".
<code>comparisons</code>	string or list	Comparison layout: "control_vs_each_disease" or explicit pairs.
<code>regulatory</code>	object	Optional regulatory metadata; <code>primary_analyte</code> seeds analyte profile merge.

12.4 The cohort hierarchy

The `controls` and `diseases` blocks define the study cohort structure. Each block has a `label` and a `groups` list. For hierarchical multiclass designs, disease groups can have `stages`:

```
{
  "controls": {
    "label": "healthy",
    "groups": [
      { "label": "all", "sample_paths": ["data/healthy.csv"] }
    ]
  },
  "diseases": {
    "label": "cancer",
    "groups": [
      {
        "label": "pca",
        "stages": [
          { "label": "pca1", "sample_paths": ["data/pca1.csv"] },
          { "label": "pca2", "sample_paths": ["data/pca2.csv"] }
        ]
      }
    ]
  }
}
```

```

    }
  ]
}
}

```

methyl-centroid builds one centroid per leaf group; **methyl-detector** compares control leaves versus each disease stage. Resolved group labels for Monte Carlo and predictors concatenate parent and leaf names (`pca_pca1`, ...).

12.5 Where tool parameters live: `actionConfig`

Parameters previously embedded in `step_config` now live under `actionConfig.<action>` in pipeline profiles and optionally the site manifest:

<code>actionConfig</code> key	Handled by	Description
<code>centroid</code>	<code>methyl-centroid</code>	Coverage thresholds, histogram bins, GPU, parallelism.
<code>detection</code>	<code>methyl-detector</code>	Statistical test, effect size, DMP export mode.
<code>dmp_selection</code>	<code>methyl-dmp-select</code>	FeatureCuts BA gate, panel size bounds.
<code>mapper</code>	<code>methyl-mapper</code>	GTF, gene window sizes, enrichment source.
<code>enricher</code>	<code>methyl-enricher</code>	Library selection, disease filters, module scoring.
<code>classifier</code>	<code>methyl-classifier</code>	Chromosome weighting, OvR layout, panel definition.
<code>gene_selection</code>	<code>methyl-gene-select</code>	Gene FeatureCuts caps, biomarker filter.
<code>predictor</code>	<code>methyl-predictor</code>	Panel definition, debug mode.
<code>validation</code>	<code>methyl-validation</code>	Monte Carlo settings, stability gates, backend profiles.
<code>alignment_qc</code>	<code>methyl-alignment-qc</code>	Schema validation, guardrails.

Exhaustive parameter tables: [configuration-reference](#) (keys listed as `actionConfig.*` sections).

Resolution: `resolve_action_config / resolve_for_project` in `packages/methylutils/methyl_utils/action`.

12.6 Monte Carlo configuration

Monte Carlo settings belong in **profile** `actionConfig.validation` (and optional site defaults), not in the study manifest. When `methyl-validation` or the workflow planner runs, merged validation config becomes `MonteCarloConfig` and is snapshotted to `monte_carlo_runs/queue/mc_config.json` for workers.

Example profile fragment:

```
"actionConfig": {
  "validation": {
    "train_fraction": 0.8,
    "n_iterations": 10,
    "seed": 42,
    "run_stability": true,
    "stability_dmp_freq": 0.8,
    "dmp_modeling_mode": "featurecuts",
    "gene_modeling_mode": "featurecuts",
    "dmp_featurecuts_target_ba": 0.9,
    "gene_featurecuts_target_ba": 0.88
  }
}
```

Inherited from the study manifest (not duplicated in the profile): `samples_base_path`, `output_base`, cohort structure, `chromosomes`, `contexts`.

12.7 Annotated study manifest example

Slim project file on `/work` (tool parameters in `mc_gene_fc.profile.json`, not here):

```
{
  "project_name": "Buffy_healthy_vs_PCa",
  "laboratory": "psomagen",
  "batch": "prostate buffy coat",
  "output_base": "/work/projects/prostate-cancer",
  "samples_base_path": "/work/samples",
  "chromosomes": ["1", "2", "21", "22"],
  "contexts": ["CG"],
  "comparisons": "control_vs_each_disease",
  "regulatory": {
    "primary_analyte": "buffy_coat",
    "stage": "feasibility"
  },
  "controls": {
    "label": "healthy",
    "groups": [{ "label": "all", "sample_paths": ["data/healthy_b.csv"] }]
  },
  "diseases": {
    "label": "cancer",
    "groups": [{ "label": "pca", "sample_paths": ["data/pca_b.csv"] }]
  }
}
```

Run with a profile:

```
methy1-workflow-run \
  --program workflow_engine/domain/checks/buffy_healthy_vs_pca/configs/buffy_mc_stability.prog
  --context-file workflow_engine/domain/profiles/mc_gene_fc.profile.json \
  --context '{"projectPath":"/work/projects/prostate-cancer/configs/project_Buffy_healthy_vs_P
```

12.8 Output directory layout

Given `output_base = "/work/projects/prostate-cancer"` and `project_name = "Buffy_healthy_vs_PCa"`:

```
/work/projects/prostate-cancer/
  Buffy_healthy_vs_PCa/
    centroids/
    detections/
      all/<comparison>/dmeps-<chrom>-discovery.csv
      dmeps-<chrom>-selected.csv
    classifiers/
    mapper/
    enricher/
    monte_carlo_runs/
      queue/mc_config.json
      run_0001/
      stability/stable_dmeps_production.csv
      production/
```

Downstream FeatureCuts and mapper steps prefer **dmeps-*-selected.csv** (BA-gated panel). Discovery CSVs remain for exploration and raw-pool modeling modes.

12.9 Related

- Usage: [Project config and layout](#)
- Reference: [Config parameter matrix](#)
- Architecture: [Layer model](#)

13 The Two Workflows

13.1 Overview

MethylPipeline supports two primary end-to-end operational workflows. They are designed to be run sequentially: Workflow 1 produces the final production model; Workflow 2 evaluates that model on random holdouts.

	Workflow 1	Workflow 2
Goal	Build a stable, production-ready model	Evaluate the frozen production model
Flag	<code>--stability</code> , then <code>--freeze</code> , then <code>--model</code>	<code>--predictor-only</code>
Steps run per iteration	centroid → detector	predictor only
Steps run once (freeze/model)	freeze/analysis: centroid → detector(fixed) → mapper → enricher (optional progression); model: classifier → predictor	—
Output	stable panel, production biological outputs, final classifier/predictor outputs	<code>all_metrics.csv</code> , <code>metrics_summary.json</code>
Answers	Which positions and model best separate the classes on all data?	What performance distribution does the frozen model achieve?

Both workflows are controlled by the study manifest (cohorts, paths, comparisons) plus the **pipeline profile** `actionConfig.validation` block (see Chapter 12). Workers receive merged settings as `resolvedConfig`; MC runs snapshot them to `monte_carlo_runs/queue/mc_config.json`.

13.1.1 DMP and gene modeling modes

Profile `actionConfig.validation` exposes `dmp_modeling_mode` and `gene_modeling_mode` (`discovery_only`, `featurecuts`, or `raw_pool`). These control which DMP CSV glob downstream steps consume:

Mode	Mapper / stability input	Typical use
<code>discovery_only</code>	<code>dmps-*-discovery.csv</code>	Broad biology, mapper/enricher exploration
<code>featurecuts</code> (default in MC profiles)	<code>dmps-*-selected.csv</code>	BA-gated prediction panel + gene FeatureCuts
<code>raw_pool</code>	Full detector export pool	Legacy / experimental paths

Gene FeatureCuts and stability aggregation follow the same mode contract. See Chapter 12 for profile examples (`dmp_modeling_mode`, `gene_modeling_mode`, FeatureCuts target BA).

13.2 Workflow 1: Model Creation

13.2.1 Theoretical Motivation

A naive approach to building a classifier would be to detect DMPs on all available samples, then evaluate the same classifier on held-out subsets. This conflates feature selection with performance estimation: the DMP panel was implicitly chosen to fit all samples, making any leave-out evaluation optimistic.

MethylValidation addresses this by repeating only the discovery portion of the pipeline on random train/validation splits. Each iteration rebuilds centroids and re-runs detection on the training cohort only. The resulting detector-side balanced accuracy summaries are used as internal estimates of discovery robustness and to filter stability runs when requested.

After many such iterations, a DMP that appears consistently in the discovery sets of most runs is more likely to reflect a true biological signal than one that appears in only a few. This motivates the stability analysis: the stable panel is a consensus over repeated random partitions of the data.

The production freeze then trains a final classifier on **all available data** restricted to the stable panel. Because the panel itself was selected using the full sample set (via the MC iterations), this is still an internal analysis; it is not a substitute for external validation on an independent cohort.

13.2.2 The Three Stages of Workflow 1

Workflow 1 is easiest to reason about as three explicit stages with different statistical roles.

13.2.2.1 Stage A — Stability (`--stability`)

Theoretical role. This stage estimates *feature-selection stability* rather than final predictive performance. Let $\mathcal{D}_{\text{disc},r}$ be the discovery DMP set from split r . A locus-level recurrence estimator

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}[d \in \mathcal{D}_{\text{disc},r}]$$

acts as a Monte Carlo approximation to selection probability under re-sampling. Thresholding with $\hat{f}(d) \geq \tau$ (where $\tau = \text{stability_dmp_freq}$) enforces reproducibility pressure on the panel.

Practical role. This stage answers: “Which loci survive repeated train/validation perturbations?” In practice:

- it runs only `centroid -> detector` per split (fast enough for many iterations),
- it creates `all_metrics.csv/metrics_summary.json` for detector-side internal diagnostics,
- it writes `stability/stable_dmps_production.csv`, which is the only accepted panel input for `--freeze`,
- and it allows optional run-quality filtering via `stability_min_balanced_accuracy` before recurrence counting.

For most projects, this is the stage where panel size and robustness are tuned (`n_iterations`, `train_fraction`, `stability_dmp_freq`).

When the iteration count is high and a single host is too slow, the same stability stage can be **executed in parallel** across multiple machines, as long as all workers share one output tree; see § 14.7 (and the user manual chapter on distributed MethylValidation).

13.2.2.2 Stage B — Freeze/Analysis (`--freeze`)

Theoretical role. This stage converts the consensus panel into a deterministic production artifact. The key operation is a *panel-constrained rerun on all samples*: detector inference is no longer discovering a new panel; it is applying fixed loci and producing full downstream biological interpretation from a stable feature space.

Practical role. This stage answers: “Given a locked panel, what is the complete biological analysis on the full cohort?” In practice it:

1. injects `fixed_dmp_panel` into `production/project.json`,
2. rebuilds centroids on all data,
3. runs detector in fixed-panel mode,
4. runs mapper and enricher to produce gene/pathway outputs,
5. optionally runs progression synthesis for ordered disease-stage summaries.

The freeze artifacts (`production/project.json`, fixed-panel detection outputs, mapper/enricher summaries) become the canonical handoff package for model release and review.

13.2.2.3 Stage C — Biological Gate + Modeling (biological-readiness, then --model)

Theoretical role. This stage first verifies biological/operational readiness from freeze artifacts, then fits the final prediction function conditioned on the frozen panel and frozen production project. It separates *feature agreement* (handled in Stage A/B) from *final classifier parameter estimation* (handled here).

Practical role. This stage answers: “Are we ready to promote the frozen artifacts to final model training, and which trained artifacts should be deployed?” In practice:

1. Run `methyl-validation biological-readiness <project_root>` to chain:
 - `methyl-enricher verify-complete`
 - `methyl-disease-progression --strict-missing`
 - `methyl-stability-freeze-readiness`
2. If readiness passes, run `--model` using profile `actionConfig.validation.model_backend`:
 - `ecdf` (default): runs `methyl-classifier` then `methyl-predictor`,
 - `tabular_sklearn`: runs model bundle -> tabular training -> tabular predictor evaluation,
 - `generative_hybrid`: runs model bundle -> latent-density generative training -> generative predictor evaluation.

For all backends, the optional biological gate (`require_biological_review_for_model`) is enforced before model creation.

`biological-readiness` runs deterministic checks regardless of Grok/xAI availability. Advisory Grok commentary is **off by default** in the chained command and enabled with `--grok-review`.

The resulting files under `production/classifiers/` and `production/predictors/` are the deployable model artifacts. Any rerun with changed panel or freeze inputs should be treated as a new model version.

13.2.3 Steps

Step 1: Prepare the profile and study manifest

Ensure the selected profile defines `actionConfig.validation` with at least `train_fraction`, `n_iterations`, `seed`, and `run_stability: true`. The study manifest supplies cohorts and paths only:

```
"actionConfig": {
  "validation": {
    "train_fraction": 0.8,
    "n_iterations": 50,
    "seed": 42,
    "run_stability": true,
    "stability_dmp_freq": 0.7,
    "stability_min_balanced_accuracy": null,
    "abort_on_step_failure": false
```

```
}  
}
```

Pass the profile via workflow context (pipelineProfile) or METHYL_PROFILE.

Step 2: Run Monte Carlo with stability

```
source .venv/bin/activate  
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --stability
```

This runs `n_iterations` stratified train/validation splits. Per iteration:

1. `methyl-centroid --group group1` and `--group group2` build separate centroid H5 files for the training split.
2. `methyl-detector --project` detects DMPs between the training centroids.

After all iterations, stability analysis counts DMP recurrence across the discovery sets:

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}[d \in \mathcal{D}_{\text{disc},r}]$$

and writes `monte_carlo_runs/stability/stable_dmps_production.csv`.

Step 3: Run the production freeze

```
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --freeze
```

This calls `freeze_production_model()` and then (optionally) progression synthesis which:

1. Merges stable DMP CSVs into `monte_carlo_runs/production/stable_dmps_genomewide.csv`.
2. Creates `monte_carlo_runs/production/project.json` with `fixed_dmp_panel` injected into resolved `actionConfig.detection`.
3. Runs production freeze on **all samples**: `centroid` $\rightarrow$ `detector(fixed panel)` $\rightarrow$ `mapper` $\rightarrow$ `enricher`.
4. If profile `actionConfig.progression.enabled=true`, runs `methyl-disease-progression` to aggregate cross-stage mapper/enricher outputs.
5. Writes `monte_carlo_runs/production/production_summary.json`.

Step 4: Run biological readiness gate

```
methyl-validation biological-readiness /path/to/<project_root> --grok-review
```

Use `--grok-review` only when you want advisory LLM commentary in addition to deterministic verdicting.

Step 5: Run final model build

```
methylation-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --model
```

This runs `run_pipeline_for_model()`:

1. `methylation-classifier --project production/project.json`
2. `methylation-predictor --project production/project.json`

If profile `actionConfig.validation.require_biological_review_for_model=true`, `--model` is blocked until `biological_review_confirmed=true`.

13.2.4 Workflow 1 Diagram

13.2.5 Expected Outputs

After Workflow 1 completes:

```
monte_carlo_runs/  
  run_0001/ ... run_000N/      ← per-iteration outputs  
    detections/**/result*.json  
  all_metrics.csv              ← one row per iteration  
  metrics_summary.json         ← mean, std, percentiles per metric  
  stability/  
    stable_dmps_production.csv  
    stability_summary.json  
  production/  
    project.json               ← frozen project with fixed_dmp_panel  
    centroids/                 ← built on all samples  
    detections/                ← using fixed DMP panel  
    mapper/  
    enricher/  
    progression/               ← optional (when enabled)  
    classifiers/               ← created by --model  
    predictors/                ← created by --model
```

13.3 Workflow 2: Model Use for Prediction

13.3.1 Theoretical Motivation

Once a production model exists, a natural question is: what balanced accuracy does it achieve on random holdouts drawn from the available cohort? This is not the same as the BA measured in Workflow 1, because Workflow 1 re-trained the model on each split while Workflow 2 applies the frozen model.

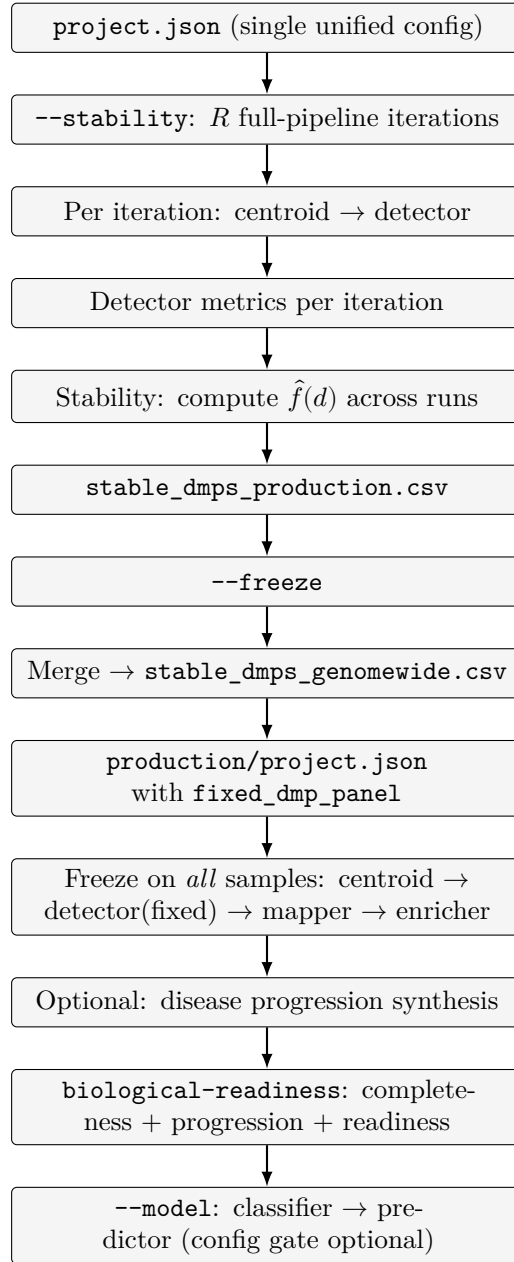


Figure 13.1: Workflow 1: Model Creation — from Monte Carlo splits to freeze, biological gate, and model build.

Workflow 2 provides the empirical distribution of the frozen model’s performance under random sampling. It is faster than Workflow 1 (only the predictor runs per iteration), and it answers the question: *is the production model’s accuracy stable across different random splits, or does it depend heavily on which specific samples are held out?*

Important caveat. The available cohort used in Workflow 2 overlaps with the training data for the production model (the production model was trained on all samples). This means Workflow 2 is not a hold-out evaluation in the classical sense; the test samples may also be in the training centroid. A truly external validation cohort would be needed to measure generalization to unseen data.

13.3.2 Steps

Step 1: Confirm the production model exists

The file `monte_carlo_runs/production/project.json` must exist (created by Workflow 1’s `--freeze` step).

Step 2: Run predictor-only evaluation

```
methy-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --predictor-only
```

Per iteration, this:

1. Creates a new split-specific `project.json` with train/validation CSVs.
2. Merges the production model paths from `production/project.json` into the run-specific project (via `apply_frozen_pipeline_artifacts_to_run_project()`).
3. Runs only `methy-predictor` on the validation split.
4. Writes `validation_metrics.json`.

After all iterations, the same aggregation as Workflow 1 produces `all_metrics.csv` and `metrics_summary.json`.

13.3.3 Workflow 2 Diagram

13.4 Workflow 3: True Held-Out Batch Evaluation

13.4.1 Theoretical Motivation

Workflow 2 answers “how stable is the frozen model across random holdouts drawn from the available cohort?”, but its holdouts overlap the training centroid, so it is not a hold-out evaluation in the classical sense (§ 13.3). Workflow 3 closes that gap: it evaluates the frozen production model on a **designated batch of samples that were never used for training or model selection**, and characterizes the *sampling distribution* of the quality metrics by bootstrap.

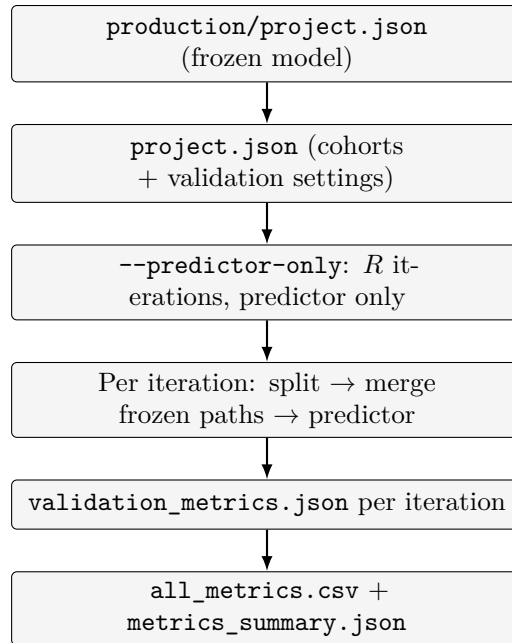


Figure 13.2: Workflow 2: Model Use for Prediction — predictor-only evaluation of the frozen model.

This is the same reporting philosophy used by MethyIIT ([Sanchez et al. 2019](#)), which evaluates classifier performance with Monte Carlo resampling and reports each indicator with bootstrap confidence intervals rather than a single point estimate.

13.4.2 Designating the Held-Out Batch

Held-out samples are declared through the `validation_partitions` contract in `profile/project actionConfig.validation` (the same object used for lifecycle governance). The default hold-out role is `locked_test`; `holdout_partition` selects a different role (for example `pivotal_validation` or `post_market_monitoring`).

The hold-out is **stratified per class**: holding out 10% removes 10% of *each* group independently, so class balance is preserved in both the hold-out and the remaining active pool. The helper `methyl_validation.holdout_eval.stratified_holdout_by_fraction(cohorts, fraction, seed=...)` produces such a split (returning `{label: {"holdout": [...], "active": [...]}}`); its `holdout` lists populate the partition. The **active** remainder (e.g. 90%) is what `--stability` consumes, and `stability` then further partitions the active pool into its own train/validation splits (for example 80/20 per iteration). The hold-out samples never enter `stability` or the final freeze.

```

"actionConfig": {
  "validation": {
    "validation_partitions": {
      "locked_test": ["batch7/S001", "batch7/S002", "batch7/S003"]
    },
    "holdout_partition": "locked_test",
    "holdout_n_bootstrap": 1000,
  }
}

```

```

    "holdout_ci": 0.95,
    "holdout_stratified": true
  }
}

```

The partition contract already validates that hold-out samples do not overlap `development_train` at config-parse time. The held-out samples must also appear in the project's control/disease cohorts (with their true labels) so that each hold-out sample can be assigned a class for labeled metrics.

13.4.3 Disjointness Guarantee (Exclusion at Freeze)

For the held-out claim to be valid, the frozen model must not have been trained on the hold-out samples at **any** stage. When `holdout_exclude_from_training=true` (default):

- `--stability` excludes the hold-out partition from the Monte Carlo cohort pool, so feature selection (DMP discovery) sees only the active samples;
- `--freeze` filters the hold-out samples out of the production cohort list files before building centroids/detector/classifier, and records the excluded set together with each sample's recovered class label in `monte_carlo_runs/production/holdout_manifest.json`.

Workflow 3 preflights that manifest and refuses to run if any current hold-out sample was not excluded, so a false held-out claim cannot be produced silently. Because freeze removes the hold-out samples from the cohort CSVs, their class labels can no longer be read from those files; Workflow 3 therefore recovers labels from the manifest's `holdout_class_map` recorded at freeze time.

13.4.4 Bootstrap Distribution of QC Metrics

The frozen model scores the hold-out batch **once** (a single `methyl-predictor` pass), producing per-sample true labels, predictions, and class probabilities. The per-sample results are then resampled (stratified by class by default) `holdout_n_bootstrap` times. For each resample the following are recomputed:

- balanced accuracy, sensitivity, specificity, F_1 , ROC-AUC (binary), or
- balanced accuracy, macro recall, macro specificity, macro F_1 , macro one-vs-rest AUC, plus control-vs-pooled-disease screening sensitivity/specificity/ F_1 /AUC (multiclass).

Each metric is summarized by its mean, standard deviation, two-sided bootstrap confidence interval at level `holdout_ci`, and percentiles. ROC-AUC is reported as NaN (and excluded from a summary) for any resample in which only one class is present.

13.4.5 Steps

```

source .venv/bin/activate
# 1. Build the frozen model with hold-out excluded from training:
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --stability
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --freeze
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --model
# 2. Evaluate the frozen model on the designated hold-out batch:
methyl-validation --project configs/project_Healthy_vs_PCa1-4-CG.json --holdout-eval

```

Outputs under `monte_carlo_runs/holdout_batch/`:

- `holdout_metrics_bootstrap.json` — point estimate plus per-metric distribution summary,
- `holdout_metrics_distribution.csv` — one row per bootstrap resample (for custom plots),
- `holdout_metrics_distribution.html` — KDE/ECDF of each metric,
- `predictor/` — the raw per-sample predictions and `validation_metrics.json`.

13.4.6 Caveat

Bootstrapping characterizes uncertainty due to the finite size and composition of the *provided* hold-out batch. It is not a substitute for prospective external validation on an independent population; it quantifies how much the reported metrics would vary if the same held-out batch had been sampled slightly differently.

13.5 Diagnostic: DMP Coverage

Symptom. `sample_dmp_coverage.dmps_used_fraction_median < 0.5` in `validation_metrics.json`. Values below 0.3 indicate severe degradation.

Cause. The centroid/detector `min_coverage` (e.g. 4) is lower than the **MethylClassifier** per-locus coverage requirement for new samples, so many DMP loci are treated as missing in the feature matrix.

Fix. In profile `actionConfig.classifier` (MethylClassifier config), set `min_coverage` to match centroid `base_config.min_coverage` (see classifier documentation for the exact field name in your layout).

Verification. Re-run Workflow 2. `dmps_used_fraction_median` should rise to 0.6–0.9 for typical WGBS data.

13.6 Diagnostic: Class Imbalance

Symptom. The confusion matrix shows near-perfect recall for the majority class (e.g. healthy/control) and near-zero recall for minority classes (disease stages). Balanced accuracy is close to $1/K$ (random chance).

Cause. The training cohort has substantially more samples in one class. For example, 86 healthy vs 25–36 disease samples per stage (41% healthy in the example project). The multinomial logistic regression learned head defaults to the majority class when the histogram scores are ambiguous (which is common when DMP coverage is low).

Fix. In profile `actionConfig.detection`, add:

```
"multiclass_train_learned_head": true,  
"multiclass_learned_class_weight": "balanced"
```

The `balanced` setting computes $w_k = n_{\text{total}} / (K \cdot n_k)$, penalizing majority-class errors proportionally less than minority-class errors. See § 5.9 for the full derivation.

13.7 Diagnostic: Sklearn Version Mismatch

Symptom. Warning messages during prediction: `InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version X.Y.Z when using version A.B.C.`

Cause. The `multiclass-classifier.pkl` was serialized with a different version of scikit-learn than the one currently installed.

Diagnosis. Check the `sklearn_version` field in the model PKL metadata:

```
import pickle  
with open("monte_carlo_runs/production/classifiers/multiclass-classifier.pkl", "rb") as f:  
    model = pickle.load(f)  
print(model["metadata"].get("sklearn_version")) # version used at training time
```

Compare to the current installed version:

```
import sklearn; print(sklearn.__version__)
```

Fix. Re-run the `--freeze` step with the current sklearn version. The logistic regression head will be retrained and the new PKL will record the correct version. New model PKLs produced after the fix always record `sklearn_version` in their metadata.

13.8 Choosing Workflow Parameters

13.8.1 `train_fraction`

Higher values (0.8–0.9) leave less data for validation but produce more powerful centroids in training. For small cohorts (total $n < 50$ per class), prefer 0.8. For large cohorts, 0.9 is reasonable.

13.8.2 `n_iterations`

More iterations reduce the variance of the empirical performance summaries. 20–50 iterations is typically sufficient to characterize the mean and standard deviation of balanced accuracy. 100+ iterations are useful for stable percentile estimates (e.g. p5, p95).

13.8.3 `stability_dmp_freq`

Higher values (0.8–1.0) produce smaller, more reproducible panels but may miss positions that are biologically informative but variable across splits. Lower values (0.5–0.7) retain more positions at the cost of including split-sensitive noise. A good starting point is 0.7.

13.8.4 `stability_min_balanced_accuracy`

Setting this parameter (e.g. to 0.6) excludes iterations with poor validation performance from the DMP frequency count. This can improve panel quality by excluding outlier runs where the centroid construction or detection failed to find informative DMPs. It also reduces the effective R^* and thus increases the variance of $\hat{f}(d)$. Use with caution for small `n_iterations`.

14 Model Creation and Validation

This chapter consolidates the implementation-facing view of model training and validation across the current MethylPipeline stack.

It complements:

- § 13.2 (workflow theory),
 - § 13.3 (predictor-only evaluation),
 - Usage manual ch.05–08 for operator commands ([docs/usage/](#)).
-

14.1 Scope

Model creation in MethylPipeline has two layers:

1. **Biology-first feature stabilization** (Monte Carlo stability + freeze), and
2. **Final model fitting and evaluation** (`--model` backend path).

The validated production sequence is:

- `--stability` (MC centroid -> detector recurrence),
 - `--freeze` (single all-sample run with fixed panel + interpretation),
 - `--model` (backend-specific train/predict),
 - optional `--predictor-only` for repeated frozen-model performance distribution.
-

14.2 Training and Validation Lifecycle

```
flowchart TD
  A[Project config + cohort definitions]
  B[Stage stability: MC centroid to detector]
  C[Stable panel from recurrence thresholding]
  D[Stage freeze: all-sample run with fixed panel]
  E[mapper enricher progression optional]
  F[Stage model: backend training + predictor]
  G[Production metrics + model artifacts]
```

```
H[Optional repeated frozen-model predictor evaluation]
A --> B --> C --> D --> E --> F --> G --> H
```

For CLI entry points and artifact gates, see Usage ch.05–08.

14.3 Backend Roles

Current `--model` backend semantics:

- **ecdf**: `methyl-classifier` -> `methyl-predictor` (Bayesian/ECDF path, DMP-centric).
- **tabular_sklearn**: model bundle -> tabular train -> tabular predictor.
- **generative_hybrid**: model bundle -> latent generative train -> generative predictor.

Covariate fusion is backend-specific:

- **ecdf**: DMP-only reference path.
- **tabular_sklearn** / **generative_hybrid**: DMP features plus optional covariates.

14.4 Covariate-Aware Validation Contract

When covariates are enabled for tabular/generative backends:

- join key is sample basename against `covariate_id_column`,
- numeric/ordinal/categorical role assignment is fixed at training and reused at inference,
- numeric imputation + optional standardization are frozen from training statistics,
- ordinal columns use preserved label->code mappings (`covariate_ordinal_maps`) with unknown-value fallback,
- categorical encoding uses frozen vocabulary with unknown-category handling,
- shape/schema consistency is enforced before prediction.

This contract keeps training and inference feature spaces aligned while preserving reproducibility across reruns.

14.5 Outputs to Review

For model readiness decisions, review at least:

- `monte_carlo_runs/stability/stable_dmps_production.csv`,
- `monte_carlo_runs/production/production_summary.json`,
- `monte_carlo_runs/production/classifiers/*`,
- `monte_carlo_runs/production/predictors/validation_metrics.json`,
- `monte_carlo_runs/all_metrics.csv` and `metrics_summary.json` (or predictor-only equivalents).

For covariate-aware backends, also review backend metadata and preprocessing artifacts under the production classifier directory.

14.6 Configuration Audit for Accuracy-First Modeling

The latest workflow (`--stability -> --freeze -> --model` with optional `--predictor-only`) is now mature enough that some configuration surfaces can be simplified without losing capability.

14.6.1 Deprecation candidates (proposed)

These are practical candidates for staged deprecation in a future major release:

Candidate key / pattern	Current status	Why deprecate	Replacement
<code>actionConfig.validationBlockMapperAndEnricher</code>	Marked deprecated in config reference	Stability iterations now run centroid+detector only; mapper/enricher belong to --freeze	Remove flag; always run mapper/enricher in --freeze when enabled in project steps
<code>actionConfig.validationBlockSkipEnricherToRunMapperAndEnricher</code>	Marked deprecated	Semantics overlap with step-level enricher control and creates branchy MC behavior	Drop from validation block; control enricher directly in freeze-stage execution policy
<code>actionConfig.validator</code> (alias)	Deprecated alias for <code>actionConfig.predictor</code>	Duplicates predictor namespace and complicates resolver logic	Use only <code>actionConfig.predictor</code>

Candidate key / pattern	Current status	Why deprecate	Replacement
Flat cohort schema (<code>group1 / group2 / top-level groups</code>)	Backward-compatible path	New control/disease/comparisons schema is clearer and supports multiclass/stage workflows natively	Use <code>control</code> , <code>disease</code> , and explicit <code>comparisons</code>

Deprecation strategy recommendation:

1. Keep warnings in the next minor release.
2. Add a migration helper command that rewrites legacy project JSON fields.
3. Remove legacy keys in the next major version after one full release cycle.

14.6.2 Accuracy-first project profile

For the best downstream predictive performance, favor stable panel discovery first, then conservative production fitting:

- **Stability stage quality:** use enough iterations (`n_iterations` $\geq$ 50) and apply run filtering (`stability_min_balanced_accuracy`) only when sample count supports it.
- **Coverage consistency:** keep centroid, detector, and `MethylClassifier` `min_coverage` aligned so DMP loci visible during discovery stay observable in new samples.
- **Panel robustness over aggressiveness:** use detector `FeatureCuts` in stability/freeze (`classifier_dmp_selection="featurecuts_validation"`) with a minimum panel size floor (`min_selected_dmps`) so high BA does not come from brittle tiny panels.
- **Generalization-first model selection:** prioritize `optimization_validation` (held-out fold) for backend and parameter decisions; use `training_fold_validation` only as overfit diagnostic.
- **Covariate discipline:** only enable covariates when join coverage is high and schema is stable; otherwise default to pure ECDF path to avoid leakage/shift from weak sidecars.

14.6.3 Reference tuning template (binary ECDF-first)

```
"actionConfig": {
  "validation": {
    "train_fraction": 0.8,
    "n_iterations": 50,
    "seed": 42,
    "run_stability": true,
    "stability_dmp_freq": 0.7,
    "stability_min_balanced_accuracy": 0.6,
    "model_backend": "ecdf"
```

```

},
"detection": {
  "min_coverage": 4,
  "validation_split_ratio": 0.2,
  "validation_n_repeats": 5,
  "classifier_dmp_selection": "featurecuts_validation",
  "min_selected_dmps": 500,
  "dynamic_dmp_cutoff_enabled": true,
  "effect_size_coverage": 0.95,
  "enable_platt_calibration": true
}
}

```

Use this as a starting point, then tune in this order:

1. stabilize panel recurrence (`stability_dmp_freq`, `n_iterations`),
2. verify holdout-vs-training gap (`optimization_validation` vs `training_fold_validation`),
3. only then tune panel size/FeatureCuts constraints for marginal BA gains.

14.7 Distributed Monte Carlo on shared storage

Large `n_iterations` in stability discovery are often limited by **wall time** on a single host, not by statistical need. Sharded Monte Carlo requires shared read–write access to the same `output_base` tree. The queue workflow is a **plan** → **workers** → **aggregate** pattern.

Operator documentation: [Usage ch.13](#) and [packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md](#).

14.8 Optional pipeline hyperparameter search

The package implements a read-only **scalar objective** (`J()`) from completed runs, combining weighted validation metrics with optional panel-size terms. Discovery/stability optima and calibration metrics can **conflict**; a **two-stage** search is often safer than a single ad hoc (`J`).

Theory and formulas: [packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md](#). **Operator steps:** [Usage ch.15](#).

Blondel, Vincent D., Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. “Fast Unfolding of Communities in Large Networks.” *Journal of Statistical Mechanics: Theory and Experiment* 2008 (10): P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>.

- Chen, Edward Y., Christopher M. Tan, Yan Kou, et al. 2013. “Enrichr: Interactive and Collaborative HTML5 Gene List Enrichment Analysis Tool.” *BMC Bioinformatics* 14: 128. <https://doi.org/10.1186/1471-2105-14-128>.
- DerSimonian, Rebecca, and Nan Laird. 1986. “Meta-Analysis in Clinical Trials.” *Controlled Clinical Trials* 7 (3): 177–88.
- Domingos, Pedro, and Michael Pazzani. 1997. “On the Optimality of the Simple Bayesian Classifier Under Zero-One Loss.” *Machine Learning* 29 (2): 103–30.
- Feller, William. 1948. “On the Kolmogorov-Smirnov Limit Theorems for Empirical Distributions.” *The Annals of Mathematical Statistics* 19 (2): 177–89. <https://doi.org/10.1214/aoms/1177730243>.
- Fisher, Ronald A. 1925. *Statistical Methods for Research Workers*. Oliver; Boyd.
- Fritsch, F. N., and R. E. Carlson. 1980. “Monotone Piecewise Cubic Interpolation.” *SIAM Journal on Numerical Analysis* 17 (2): 238–46. <https://doi.org/10.1137/0717021>.
- Kolmogorov, A. N. 1933. “Sulla Determinazione Empirica Di Una Legge Di Distribuzione.” *Giornale Dell’Istituto Italiano Degli Attuari* 4: 83–91.
- Mann, H. B., and D. R. Whitney. 1947. “On a Test of Whether One of Two Random Variables Is Stochastically Larger Than the Other.” *The Annals of Mathematical Statistics* 18 (1): 50–60. <https://doi.org/10.1214/aoms/1177730491>.
- Ochoa, David, Andrew Hercules, Miguel Carmona, et al. 2023. “The Next-Generation Open Targets Platform: Reimagined, Redesigned, Rebuilt.” *Nucleic Acids Research* 51 (D1): D1353–59. <https://doi.org/10.1093/nar/gkac1046>.
- Piñero, Janet, Juan M. Ramírez-Anguita, Judit Saüch-Pitarch, et al. 2020. “The DisGeNET Knowledge Platform for Disease Genomics: 2019 Update.” *Nucleic Acids Research* 48 (D1): D845–55. <https://doi.org/10.1093/nar/gkz1021>.
- Platt, John C. 1999. “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods.” In *Advances in Large Margin Classifiers*. MIT Press.
- Sanchez, Roberly, Xiaodong Yang, Thomas Maher, and Sally A. Mackenzie. 2019. “Discrimination of DNA Methylation Signal from Background Variation for Clinical Diagnostics.” *International Journal of Molecular Sciences* 20 (21): 5343. <https://doi.org/10.3390/ijms20215343>.
- Storey, John D. 2003. “The Positive False Discovery Rate: A Bayesian Interpretation and the q-Value.” *The Annals of Statistics* 31 (6): 2013–35. <https://doi.org/10.1214/aos/1074290335>.
- Stouffer, Samuel A., Edward A. Suchman, Leland C. DeVinney, Shirley A. Star, and Robin M. Williams. 1949. *The American Soldier: Adjustment During Army Life*. Princeton University Press.